

Fiche n°1

Structure de base d'un document HTML5(Rappel)

Les documents HTML sont des documents en texte brut enregistrés avec une extension de fichier .html ou .htm plutôt qu'une extension de fichier .txt.

Tous les documents HTML ont une structure minimale obligatoire qui comprend la déclaration et des éléments suivant: `<!DOCTYPE html>`, `<html>`, `<head>` et `<body>`.

```
<!DOCTYPE html>

<html Lang="fr">

  <head>

    <meta charset="utf-8"/> <title> Titre de la page </title>

  </head>

  <body> Contenu visible </body>

</html>
```

La déclaration du type de document, ou `<!DOCTYPE html>`, informe les navigateurs web de la version de HTML utilisé et est placée au début du document HTML. Une déclaration pour un document écrit en HTML5 est simplement `<!DOCTYPE html>`

Après la déclaration de type de document, l'élément `<html>` marque le début du document.

A l'intérieur de `<html>`, la balise `<head>` contient l'entête du document, y compris toutes les métadonnées (informations sur la page Web). Le contenu à l'intérieur de l'élément `<head>` n'est pas affiché sur la page Web elle-même. A la place, il peut contenir entre autres :

`<title>` qui est le titre de la page Web qui apparaît dans la barre des titres du navigateur. C'est le seul élément dans `<head>` qui a un effet visuel



<link> Qui fait référence à une ressource données comme un fichier de style CSS

<meta> donne des informations supplémentaires sur la page comme, l'encodage, le résumé de la page, des instructions pour les moteurs d'indexation comme Google,

<style> délimite du code CSS directement dans la page Web

<script> permet d'inclure du code JavaScript dans la page ou bien de faire référence à un fichier contenant du code JavaScript.

Le code suivant donne un exemple de <head> sachant qu'aucun de ces éléments n'est vraiment obligatoire.

```
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Université Virtuelle Africaine">
  <meta name="keywords" content="UVA, AVU, Université Virtuelle,
Afrique">
  <meta name="author" content="UVA">
  <link rel="stylesheet" type="text/css" href="theme.css">
  <style>
    h1 {color:red;}
    p {color:blue;}
  </style>
  <script>
    document.getElementById("demo").innerHTML = "Hello JavaScript!";
  </script>
</head>
```

La dernière série de balises <body> et </ body>, entoure tout le contenu dans la page WEB qui est censé être affiché comme les titres, les paragraphes, les images, les tableaux etc. Il

a quelques attributs intéressent comme background permet de mettre une image ou une couleur en fond de la page Web.

Fiche n°2

Syntaxe de base de HTML5(Rappel)

- ❖ HTML est un langage de balisage permettant de structurer comment le contenu de la page Web va être interprété et présenté par le navigateur.
 - ❖ Toutes les balises ont le même format : elles commencent par un signe inférieur à « < » et finissent par un signe supérieur à « > ».
- Exemple de balises : <a>,
, <html>.

Les attributs des balises

- ❖ Les balises peuvent également contenir des attributs. Les attributs des balises sont des mots qui décrivent des propriétés de l'élément, et donnent des instructions au navigateur. Les attributs d'un élément HTML sont insérés dans la balise ouvrante sous la forme générale suivante :

<balise nom_attribut="valeur" ...>

- ⇒ Le nom de l'attribut est suivi d'un signe égal = et la valeur est mise entre guillemets. La ligne suivante donne quelques exemples de balises avec attributs.
- ❖ Il existe quelques attributs qui sont communs à presque toutes les balises dont les plus utilisés sont :
 - **id** : cet attribut qui permet de donner un identifiant unique à un élément sur la page Web. Sa valeur est un texte alphanumérique fourni par le programmeur. Cet attribut est presque systématiquement utilisé dans les fichiers CSS et dans le code JavaScript.
 - **style** et **class** : ces attributs permettent d'appliquer un style CSS sur un élément. Leurs contenus seront vu dans la partie CSS de ce chapitre
 - **title** : Cet attribut permet d'ajouter une information supplémentaire sur un élément. Sa valeur est souvent présentée comme un texte infobulle par le navigateur.
- ❖ Le code HTML a quelques règles qui sont importantes à connaître :
 - Insensible à la casse : HTML ne fait pas de différence entre les majuscules et les minuscules dans le nom des balises. est donc équivalent à . Toutefois dans le contenu HTML un texte en majuscule (resp. minuscule) reste en majuscule (resp. minuscule) à moins que vous ayez explicitement spécifié une transformation de casse dans le code. L'insensibilité à la casse opère dans l'interprétation des balises et de leurs attributs.
 - Balises imbriquées : Lorsque des balises sont imbriquées les uns dans les autres, ils doivent suivre une règle simple : Si une balise est ouverte, toutes les balises en pair qui sont dans son contenu doivent être fermées avant que sa balise fermante.
- ❖ Quelques balises auto fermantes:
, <embed>, <hr>, , <input>, <link>, <meta>, <param>, <source>, <wbr>

Fiche n°3

Liste des balises en HTML5

❖ Balises de premier niveau

Les balises de premier niveau sont les principales balises qui structurent une page HTML. Elles sont indispensables pour réaliser le « code minimal » d'une page web.

- **<html>** Balise principale
- **<head>** En-tête de la page
- **<body>** Corps de la page

❖ Balises d'en-tête

Ces balises sont toutes situées dans l'en-tête de la page web, c'est-à-dire entre **<head>** et **</head>** :

- **<link />** Liaison avec une feuille de style
- **<meta />** Métadonnées de la page web (charset, mots-clés, etc.)
- **<script>** Code JavaScript
- **<style>** Code CSS
- **<title>** Titre de la page

❖ Balises de structuration du texte

- **<abbr>** Abréviation
- **<blockquote>** Citation (longue)
- **<cite>** Citation du titre d'une œuvre ou d'un événement
- **<q>** Citation (courte)
- **<sup>** Exposant
- **<sub>** Indice
- **** Mise en valeur forte
- **** Mise en valeur normale
- **<mark>** Mise en valeur visuelle
- **<h1>** Titre de niveau 1
- **<h2>** Titre de niveau 2
- **<h3>** Titre de niveau 3
- **<h4>** Titre de niveau 4
- **<h5>** Titre de niveau 5
- **<h6>** Titre de niveau 6
- **** Image
- **<figure>** Figure (image, code, etc.)
- **<figcaption>** Description de la figure
- **<audio>** Son
- **<video>** Vidéo

- **<source>** Format source pour les balises <audio> et <video>
- **<a>** Lien hypertexte
- **
** Retour à la ligne
- **<p>** Paragraphe
- **<hr />** Ligne de séparation horizontale
- **<address>** Adresse de contact
- **** Texte supprimé
- **<ins>** Texte inséré
- **<dfn>** Définition
- **<kbd>** Saisie clavier
- **<pre>** Affichage formaté (pour les codes sources)
- **<progress>** Barre de progression
- **<time>** Date ou heure

❖ Balises de listes

Cette section énumère toutes les balises HTML permettant de créer des listes (listes à puces, listes numérotées, listes de définitions...)

- **** Liste à puces, non numérotée
- **** Liste numérotée
- **** Élément de la liste à puces
- **<dl>** Liste de définitions
- **<dt>** Terme à définir
- **<dd>** Définition du terme

❖ Balises de tableau

- **<table>** Tableau
- **<caption>** Titre du tableau
- **<tr>** Ligne de tableau
- **<th>** Cellule d'en-tête
- **<td>** Cellule
- **<thead>** Section de l'en-tête du tableau
- **<tbody>** Section du corps du tableau
- **<tfoot>** Section du pied du tableau

❖ Balises de formulaire

- **<form>** Formulaire
- **<fieldset>** Groupe de champs
- **<legend>** Titre d'un groupe de champs
- **<label>** Libellé d'un champ
- **<input />** Champ de formulaire (texte, mot de passe, case à cocher, bouton, etc.)
- **<textarea>** Zone de saisie multiligne
- **<select>** Liste déroulante
- **<option>** Élément d'une liste déroulante
- **<optgroup>** Groupe d'éléments d'une liste déroulante

❖ Balises sectionnantes

Ces balises permettent de construire le squelette de notre site web.

- **<header>** En-tête
- **<nav>** Liens principaux de navigation
- **<footer>** Pied de page
- **<section>** Section de page
- **<article>** Article (contenu autonome)
- **<aside>** Informations complémentaires

❖ Balises génériques

Les balises génériques sont des balises qui n'ont pas de sens sémantique.

En effet, toutes les autres balises HTML ont un sens :<p> signifie « Paragraphe », <h2> signifie « Sous-titre » etc.

Parfois, on a besoin d'utiliser des balises génériques (aussi appelées balises universelles) car aucune des autres balises ne convient. On utilise le plus souvent des balises génériques pour construire son design.

Il y a deux balises génériques : l'une est inline, l'autre est block.

- **** Balise générique de type inline
- **<div>** Balise générique de type block

Ces balises ont un intérêt uniquement si vous leur associez un attribut class, id ou style :

- **class**: indique le nom de la classe CSS à utiliser.
- **id**: donne un nom à la balise. Ce nom doit être unique sur toute la page car il permet d'identifier la balise. Vous pouvez vous servir de l'ID pour de nombreuses choses, par exemple pour créer un lien vers une ancre, pour un style CSS de type ID, pour des manipulations en JavaScript, etc.
- **style**: cet attribut vous permet d'indiquer directement le code CSS à appliquer. Vous n'êtes donc pas obligés d'avoir une feuille de style à part, vous pouvez mettre directement les attributs CSS. Notez qu'il est préférable de ne pas utiliser cet attribut et de passer à la place par une feuille de style externe, car cela rend votre site plus facile à mettre à jour par la suite.

Ces trois attributs ne sont pas réservés aux balises génériques : vous pouvez aussi les utiliser sans aucun problème dans la plupart des autres balises.

Fiche n°4

Syntaxe de base CSS 3 (Rappel)

C'est quoi le CSS ?

- ❖ Le CSS est un langage qui permet de styliser une page HTML à l'aide d'un sélecteur, d'une propriété et d'une valeur.
- ❖ Contrairement à l'HTML qui se reconnaît avec ses balises d'ouverture et de fermeture, le CSS lui, se reconnaît simplement avec ses classes, une accolade précédant ses propriétés et un double-point précédant ses valeurs et une accolade fermante.

Comment appliquer le CSS a une page HTML ?

- ❖ **Inline:** directement dans la balise html avec l'attribut style.

```
<body style="background-color:black;">
<h1 style="color:white;padding:30px;"> Tutorials </h1>
```

- ❖ **Embedded (interne):** directement dans le fichier html avec la balise <style> .

```
<head>
<style type="text/css">
  h1 {color:white; font-size: 10px;}
  .center {display: block; margin: 0 auto;}
</style>
</head>
```

- ❖ **Linked (externe):** dans un fichier .css externe et lié via la balise link.

```
<head>
<link rel="stylesheet" type="text/css" href="style.css" />
</head>
```

C'est quoi un sélecteur ?

- ❖ Un sélecteur est un "nom" qui indique à quels éléments d'une page HTML on désire appliquer une règle.
- ❖ Chaque règle CSS commence nécessairement par un sélecteur

Syntaxe pour une règle CSS ?

selecteur(s) { propriété: valeur ; propriété: valeur1, valeur2, valeur3 ; }

Il existe plusieurs méthodes pour identifier les éléments, par exemple:

- ❖ on indique le nom d'une balise.
- ❖ on indique la classe d'une balise.
- ❖ on indique l'identificateur d'une balise.
- ❖ on indique le nom d'une balise plus sa classe, etc.

Fiche n°5 Les attributs HTML class , id et les sélecteurs CSSLes attributs HTML id et class ?

- ❖ Ces deux attributs sont particuliers en HTML puisqu'ils n'ont pas été créés pour préciser le fonctionnement d'un élément HTML en particulier (ce qui est normalement le rôle de tout attribut HTML) mais vont être principalement utilisés pour cibler certains éléments HTML et leur appliquer des styles en CSS.
- ❖ Les attributs HTML class et id sont des attributs dits globaux car on peut les ajouter dans la balise ouvrante de n'importe quel élément HTML.
- ❖ Ces deux attributs vont être principalement utilisés dans un but de mise en forme : ils nous servir à appliquer des styles CSS aux éléments qui vont les contenir.

Comment Utiliser l'attribut « id » en HTML et en CSS ?

⇒ L'attribut universel **id** définit un identifiant qui doit être unique pour l'ensemble du document. Le but de cet attribut est de pouvoir identifier un élément lorsqu'on crée un lien, avec un fragment et qu'on souhaite le manipuler avec un script ou qu'on le met en forme avec CSS.

Exemple❖ **Code HTML**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link href="style.css" rel="stylesheet" />
  </head>
  <body>
    <h1 id="orange">Exemple Utilisation des id</h1>
    <p id="bleu"> Un premier paragraphe </p>
    <p id="rouge"> Un autre paragraphe avec un <a href="#">lien</a></p>
    <p id="gros">Un troisième paragraphe</p>
  </body>
</html>
```

❖ **Code CSS (fichier style.css)**

```
#orange{
  color: orange;
}
#bleu{
  color: blue;
}
#rouge{
  color: red;
}
#gros{
  font-size: 24px;
}
```

**Exemple Utilisation des id**

Un premier paragraphe

Un autre paragraphe avec un [lien](#)

Un troisième paragraphe

Résultat Page HTML

⇒ Pour cibler un **id** en particulier en CSS, on utilisera le symbole dièse (#) suivi de la valeur de l'**id** auquel on souhaite lier des styles.

Comment Utiliser l'attribut « class » en HTML et en CSS ?

- ⇒ L'un des grands intérêts de l'attribut HTML **class** est qu'un même attribut (et donc les styles CSS liés) va pouvoir être partagé par différents éléments. Cela facilite grandement la gestion des styles de nos fichiers HTML et nous permet de gagner beaucoup de temps.
- ⇒ En HTML5, l'attribut **class** peut servir dans n'importe quel élément HTML.
- ⇒ Pour cibler un **class** en particulier en CSS, on utilisera le symbole point (.) suivi de la valeur du class.

Valeurs d'attribut class

Valeur	Description
nomClass	• Spécifie un ou plusieurs noms de classe d'un élément. • Pour spécifier plusieurs classes, séparez les noms de classe par un espace <u>par exemple</u> <code></code>. Ce qui vous permet de combiner plusieurs classes CSS pour un élément HTML. Règles de nomination : • Doit commencer par une lettre de A à Z ou a-z • Peut être suivie : de lettres (A-Za-z), de chiffres (0-9), des tirets ("-") et de traits de soulignement (" _ ") • Comme tous les attributs en HTML, il est insensible à la casse

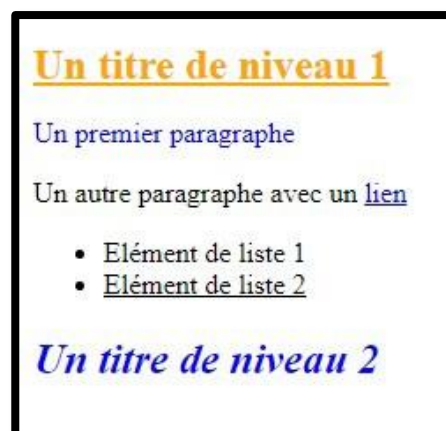
Exemple

❖ Code HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<link href="style.css" rel="stylesheet">
</head>
<body>
  <h1 class="gros souligne orange">Un titre de niveau 1</h1>
  <p class="bleu">Un premier paragraphe</p>
  <p>Un autre paragraphe avec un <a href="#">lien</a></p>
  <ul>
    <li>Elément de liste 1</li>
    <li class="souligne">Elément de liste 2</li>
  </ul>
  <h2 class="gros italique bleu">Un titre de niveau 2</h2>
</body>
</html>
```

❖ Code CSS (fichier style.css)

```
.gros{
  font-size:24px;
}
/*Texte en italique*/
.italique{
  font-style:italic;
}
/*Crée un trait de soulignement
sous le texte*/
.souligne{
  text-decoration:underline;
}
.orange{
  color:orange;
}
.bleu{ color:blue; }
```



Résultat Page HTML

Fiche n°6

Les unités et le Box Model en CSS3

Les unités en CSS ?

Les CSS proposent plusieurs unités possibles pour exprimer tailles et longueurs. Le W3C les répartit en unités absolues et unités relatives :

- ⇒ **Les unités absolues** : Il existe des unités en CSS qui ne sont influencées par aucune autre dimension. Appelées unités absolues, elles sont généralement utilisées pour traduire une longueur, par exemple un padding, une hauteur de bloc, la taille des caractères.

Les unités absolues en CSS	
px	Les pixels
in	Les pouces
cm	Les centimètres
mm	Les millimètres
pc	Les picas
pt	Les points

Exemple

```
.box {
  width : 150 px ;
  height : 150 px ;
}
```

L'élément **box** aura la même dimension (150 x 150 pixels), quelle que soit la taille de l'écran.

- ⇒ **Les unités relatives** : À l'opposé des unités absolues, il existe des unités qui font varier la taille d'un élément en fonction de la taille de la police ou de l'élément parent. Ces unités dites relatives au texte sont de plus en plus utilisées pour harmoniser les éléments d'un design. On trouve :

Les unités relatives en CSS	
em	Ou cadratin, unité proportionnelle à la taille de la police de l'élément parent ou du document
rem	L'unité rem fait toujours référence à la taille de la police de l'élément racine.
ex	Cette unité est relative à la hauteur de la police actuelle en minuscule.
ch	Cette unité est elle aussi peu utilisée, elle est relative à la largeur du caractère "0".
%	Relatives à la dimension du conteneur parent.

Par défaut, 1 em = 16 px si aucune taille de police n'est définie.

Exemple

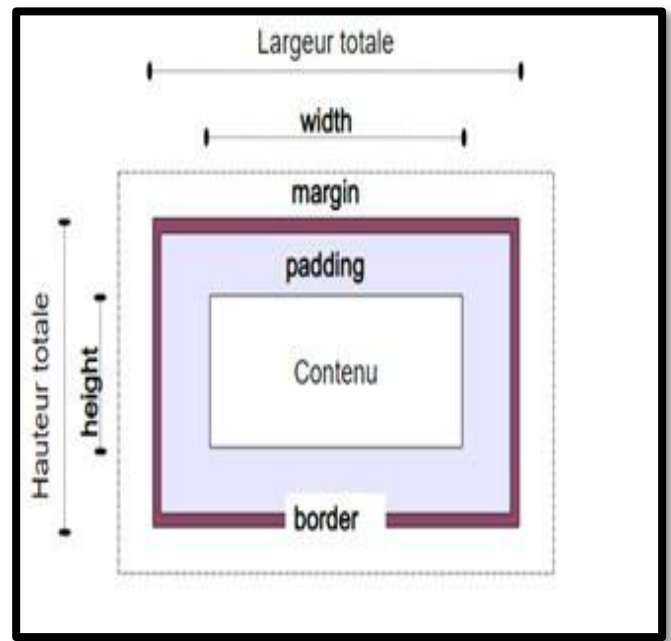
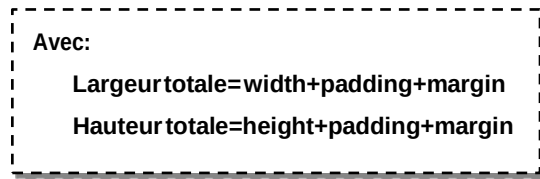
```
.box {
  width : 50%;
  Height: 50%;
}
```

Le pourcentage fera référence à la largeur calculée de l'élément parent de la boîte. Ici l'élément **box** prend 50% de la taille de son élément conteneur.

- ⇒ Les CSS offrent plusieurs autres unités comme les unités angulaires (**deg**, **grad**, **rad**, **turn**), les Unités temporelles (**s**, **ms**), les Unités de résolution (**dpi**, **dpcm**, **dppx**), les unités relatives au viewport etc...

Box Model en CSS ?

⇒ Les CSS définissent un modèle de boîte (**box model**) standard pour tous les éléments HTML.



- ❖ Le contenu est caractérisé par sa hauteur (**height**) et sa largeur (**width**).
 - ❖ On distingue deux types de marges :
 - Les marges intérieures « **padding** »
 - Les marges extérieures « **margin** »
 - ❖ Le **padding** et le **margin** sont séparés par une bordure (**border**).
 - ❖ Le **padding** est l'espace entre la bordure et le contenu html réel de l'élément.
 - ❖ Le **margin** est l'espace entre la bordure de l'élément et le reste de contenu du page web.
- ⇒ Chaque dimension (hauteur, largeur, marges) peut être traitée de manière indépendantes grâce aux mots clés : **top**, **right**, **bottom**, **left**.

Exemple:

```
.box { margin-left: 1em;
      margin-bottom: 1em;
      border-right: 1em;
}
```

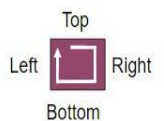
⇒ On peut utiliser la forme courte pour définir les dimensions (**margin** et **padding**) en respectant le sens de rotation horaire

Exemple

```
.box {
  margin-top: 1em;
  margin-right: 0;
  margin-bottom: 2em;
  margin-left: 0.5em;
}
```



```
.box {
  margin: 1em 0 2em 0.5em;
}
```



Types Box Model en CSS ?

⇒ Les CSS offrent plusieurs types de Box Model parmi eux on peut citer.

1) Type **inline** (boîte en ligne)

Pour une boîte de type inline les propriétés **width** and **height** ne sont pas stylables, la propriété **margin** n'aura d'effets que sur les marges gauches et droites.

2) Type **Block** (boîte en bloc)

Toutes les propriétés sont stylables (modifiables).

Exemple Box model

box.html

```
<!DOCTYPE html>
<head>
  <link href="stylebox.css" rel="stylesheet">
</head>
<body>
  <div class="main">CSS Box-Model </div>
  <!-- Class main qui contient le titre -->
  <div id="box">Exemple Box Model</div>
  <!-- id box pour le model box -->
</body>
</html>
```

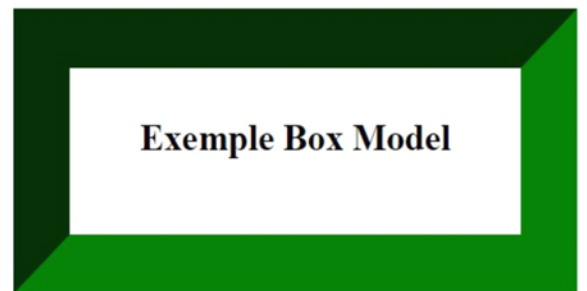
stylebox.css

```
.min {
  font-size: 32px;
  font-weight: bold;
  text-align: left;
}

#box {
  padding-top: 40px;
  width: 400px;
  height: 100px;
  border: 50px inset green;
  margin: 50px;
  text-align: center;
  font-size: 32px;
  font-weight: bold;
}
```

➡ Résultat

CSS Box-Model



Création d'un site web interactif

Fiche n°7

JavaScript

→ C'est quoi le JavaScript?

Le JavaScript est un langage de script qui offre de nombreuses possibilités. Sa présence croissante sur le web permet de naviguer sur des sites de plus en plus interactifs et rapides.

Il permet, entre autres, d'introduire sur une page web ou HTML des petites animations ou des effets. Le langage JavaScript se distingue des langages serveurs par le fait que l'exécution des tâches est opérée par le navigateur lui-même, sur l'ordinateur de l'utilisateur, et non sur le serveur web. Il s'active donc généralement sur le poste client plutôt que côté serveur.

→ Quels sont les principaux usages de JavaScript ?

Le langage JavaScript est principalement employé pour améliorer l'ergonomie d'un site Internet et/ou d'une interface applicative utilisateur. Il sert également à :

- ❖ Stocker des valeurs utiles dans des variables.
- ❖ Faire des opérations sur des chaînes de texte.
- ❖ Écouter les événements (comme un clic et mouvement de la souris)
- ❖ Modifier le HTML et le CSS de votre page après le chargement de la page
- ❖ Bouger les choses sur l'écran de manière interactive.
- ❖ Créer des jeux géniaux qui fonctionnent dans le navigateur
- ❖ Communiquer des données entre le serveur et le navigateur
- ❖ Permettre d'interagir avec une webcam, un microphone et d'autres appareils

→ Ordre d'exécution du JavaScript!

Quand le navigateur rencontre un bloc de JavaScript, il l'exécute généralement dans l'ordre, de haut en bas.

→ Comment ajouter du JavaScript à votre page?

Le JavaScript est appliqué à la page HTML un peu comme le CSS. Les éléments `<link>` permettent d'appliquer des feuilles de style externes au HTML, le CSS interne utilise l'élément `<style>`. Pour le JavaScript il y a une seule façon, avec l'élément `<script>`.

❖ JavaScript externe :

Généralement, un script est écrit dans un fichier « `.js` » à part. Pour exécuter un script depuis un fichier dans la page web, il suffira d'utiliser `<script>` avec un attribut « `src` » pointant vers le fichier du script en utilisant l'URL du fichier

```
<script src="chemin/vers/le/script.js"></script>
```

❖ JavaScript interne :

Il est également possible d'insérer du code JavaScript directement dans la balise `<script>` sans fournir d'attribut « `src` » .

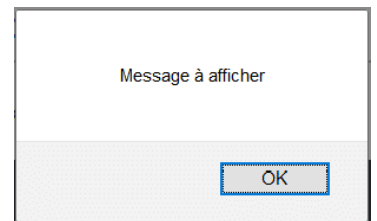
Exemple

```
<script>
window.addEventListener('load', function () {
    console.log('Cette fonction est exécutée une fois
    quand la page est chargée. ');
});
</script>
```

Pour éviter des problèmes de chargement sur les pages, il est conseillé de placer les éléments `<script>` juste avant la fermeture de l'élément `<body>`.

→ La boîte de dialogue `alert()`

`alert()` est une instruction simple, appelée fonction, qui permet d'afficher une boîte de dialogue contenant un message. Ce message est placé entre apostrophes, elles-mêmes placées entre les parenthèses de la fonction.



→ La syntaxe Javascript

La syntaxe du Javascript n'est pas compliquée. De manière générale, les instructions doivent être séparées par un point-virgule que l'on place à la fin de chaque instruction :

```
<script>
instruction_1
;
instruction_2
;
myFunction();
</script>
```

```
<script>
alert('Bonjour')
;
// la fonction affiche une boîte de
// dialogue avec "Bonjour"
</script>
```

→ Les commentaires

```
<script>
instruction_1; // Ceci est ma première instruction
instruction_2;
/* La troisième instruction ci-dessous,
   avec un commentaire sur deux lignes */
instruction_3;
</script>
```

→ Les variables en JavaScript

Une variable consiste en un espace de stockage, qui permet de garder en mémoire tout type de données. La variable est ensuite utilisée dans les scripts. Une variable contient seulement des caractères alphanumériques, le \$ (dollar) et le _ (underscore) ; elle ne peut pas commencer

par un chiffre ni prendre le nom d'une fonction existante de Javascript. On crée la variable et on lui affecte (ou attribue) une valeur :

```
<script>
    var myVariable;
    myVariable = 2;
</script>
```

ou

```
<script>
    var myVariable = 2;
</script>
```

→ Les types de variables

Une variable peut être de type numérique, une chaîne de caractères ou booléen.

```
<script>
var text = 'J\'écris mon texte ici';
/* Avec des apostrophes, le \ sert à échapper une apostrophe
intégrée dans le texte, pour ne pas que Javascript pense
que le texte s'arrête là.*/
</script>
```

→ Les opérateurs arithmétiques de variables

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (%).

→ La concaténation

Une concaténation consiste à ajouter une chaîne de caractères à la fin d'une autre, comme dans cet exemple :

```
<script>
var hi = 'Bonjour ', name = 'toi',
result;
result = hi + name;
alert(result);
// Affiche : « Bonjour toi »
</script>
```

```
<script>
    var text = 'Bonjour ';
    text += 'toi';
    alert(text);
    // Affiche « Bonjour toi ».
</script>
```

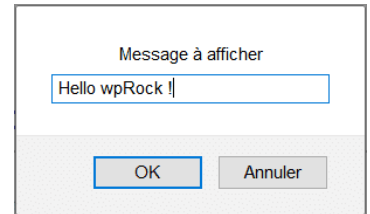
On peut convertir un nombre en chaîne de caractères avec l'astuce suivante :

```
<script>
    var text, number1 = 4, number2 = 2;
    text = number1 + '' + number2;
    /* on ajoute une chaîne de caractères vide entre les
    deux nombres pour permettre une concaténation sans
    calcul */
    alert(text); // Affiche : « 42 »
</script>
```

→ La fonction `prompt()`, avec concaténation et calcul

La boîte de dialogue `prompt()` permet d'afficher un message, un champ à remplir (input de type texte) et un bouton « OK ». Cette fonction retourne la valeur qui a été entrée dans le champ par l'utilisateur.

```
<script>
  var userName = prompt('Entrez votre prénom :');
  alert(userName);
  // Affiche le prénom entré par l'utilisateur
</script>
```



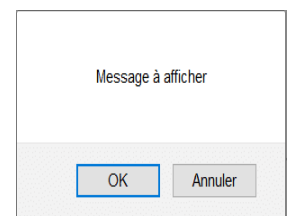
On peut aussi se servir de la fonction `prompt()` pour un calcul

```
<script>
  var first, second, result;
  first = prompt('Entrez le premier chiffre :');
  second = prompt('Entrez le second chiffre :');
  result = parseInt(first) + parseInt(second);
  /* la fonction parseInt() transforme la chaîne de
  caractères en nombre */
  alert(result);
</script>
```

→ La boîte de dialogue `confirm()`

La boîte de dialogue `confirm()` permet d'afficher un message et deux boutons : un bouton « OK » et un bouton « Annuler ». Cette fonction retourne une valeur booléenne qui vaut « true » si c'est le bouton OK qui est cliqué et retourne « false » si c'est le bouton « Annuler » qui est cliqué.

```
if ( confirm( "Message à afficher" ) ) {
  // Code à exécuter si le l'utilisateur clique sur "OK"
} else {
  // Code à exécuter si l'utilisateur clique sur "Annuler"
}
```



JavaScript Notion de base

→ Quels sont les opérateurs de comparaison à utiliser ?

==	égal à	>	supérieur à
!=	différent de	>=	supérieur ou égal à
===	contenu et type de variable égal à	<	inférieur à
!==	contenu ou type de variable différent de	<=	inférieur ou égal à

```
<script>
var number1 = 2, number2 = 2, number3 = 4, result ;
result = number1 == number2;    /* la variable result
reçoit le résultat de comparaison de contenu de deux
variables */
alert(result);    /* la condition est donc vérifiée car
les deux variables contiennent bien la même valeur*/
result = number1 == number3;
alert(result);    // la condition n'est pas vérifiée car
2 est différent de 4
result = number1 < number3;
alert(result);    // la condition est vérifiée car 2 est
bien inférieur à 4
</script>
```

→ Quels sont les opérateurs logiques à utiliser ?

- ❖ **&&** : qui signifie ET avec par exemple : **var1 && var2**. Cet opérateur vérifie la condition lorsque toutes les variables (type booléen) qui lui sont passées valent vrai (true).
- ❖ **||** : qui signifie OU avec par exemple : **var1 || var2**. Cet opérateur est plus souple car il renvoie vrai (true) si une variable qui lui est soumise contient vrai (true), qu'importe les autres variables.
- ❖ **!** : qui signifie NON avec par exemple : **!var**. Cet opérateur se différencie des deux autres car il ne prend qu'une seule valeur à la fois. S'il se nomme « NON » c'est parce que sa fonction est d'inverser la valeur qui lui est passée, ainsi true deviendra false et inversement.

Nb : **var** de type booléen ou un résultat d'une fonction qui renvoie vrai ou faux ou toute une instruction de comparaison

```
<script>
var number1 = 2, number2 = 2, number3 = 4, result;
result = (number1 < number2) && (number1==number2);
alert(result);
result = (number1 == number3) || !(number1==number2 );
alert(result);
</script>
```

→ La structure « if.. else »

La condition **if** est une structure conditionnelle limitée par définition puisqu'elle ne nous permet d'exécuter un bloc de code que dans le cas où le résultat d'un test est évalué à **true** mais elle ne nous offre aucun support dans le cas contraire.

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
alert(result);
/* La fonction alert n'affiche rien car
la condition non vérifiée*/
</script>
```

```
<script>
if (confirm('Voulez-vous exécuter le
code Javascript de cette page ?')) {
    alert('Le code a bien été exécuté !');
}
</script>
```

La structure conditionnelle **if...else** (« si... sinon » en français) va être plus complète que la condition **if** puisqu'elle va nous permettre d'exécuter un premier bloc de code si un test renvoie **true** ou un autre bloc de code dans le cas contraire.

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
else
{
    result = 'b est le plus grand';
}
alert(result);
// la fonction alert affiche "condition
pas vérifiée"
</script>
```

```
<script>
if (confirm('Pour accéder à ce site vous
devez être une fille, cliquez sur
"OK" si c\'est le cas.')) {
    alert('Vous allez être redirigé vers le
site.');
```

La condition **if...else if...else** (« si...sinon si...sinon ») est une structure conditionnelle encore plus complète que la condition **if...else** puisqu'elle va nous permettre cette fois-ci de générer et de prendre en charge autant de cas que l'on souhaite

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
else if (a<b)
{
    result = 'b est le plus grand';
}
else
{
    result = 'a et b sont egaux';
}
alert(result);
// la fonction alert affiche "b le
plus grand"
</script>
```

```
<script>
var etage = parseInt(prompt("Entrez l'étage
où l'ascenseur doit se rendre (de -2 à 30)
:"));
if (etage == 0) {
    alert('Vous vous trouvez déjà au rez-de-
chaussée.');
```

→ La structure « switch »

La structure **switch** commence par évaluer l'expression fournie (cette évaluation ne se produit qu'une fois). Si une correspondance est trouvée, le programme exécutera les instructions associées. Si plusieurs cas de figure correspondent, le premier sera sélectionné (même si les cas sont différents les uns des autres).

```
switch (expression) {  
  case valeur1:  
    // Instructions à exécuter lorsque le résultat  
    // de l'expression correspond à valeur1  
    instructions1;  
    [break;]  
  case valeur2:  
    // Instructions à exécuter lorsque le résultat  
    // de l'expression correspond à valeur2  
    instructions 2;  
    [break;]  
  ...  
  case valeurN:  
    // Instructions à exécuter lorsque le résultat  
    // de l'expression à valeurN  
    instructionsN;  
    [break;]  
  [default:]  
    // Instructions à exécuter lorsqu'aucune des  
    // valeurs  
}
```

```
switch (fruit=prompt('donner le nom de  
fruit que vous voulez'))  
{  
  case "Oranges":  
    alert ("Oranges : 2 Dt le kilo.");  
    break;  
  case "Pommes":  
    alert("Pommes : 3,5 DT le kilo.");  
    break;  
  case "Bananes":  
  case "Cerises":  
    alert ("Bananes et Cerises : 9,5 Dt le  
kilo.");  
    break;  
  default:  
    alert("Désolé, nous n'avons plus de " +  
fruit + ".");  
}
```

→ Les ternaires

L'opérateur **ternaire** conditionnel est le seul opérateur JavaScript qui comporte trois opérandes. Cet opérateur est fréquemment utilisé comme raccourci pour la déclaration de l'instructions /if...else.

```
Condition ? exprSiVrai : exprSiFaux
```

```
var age=parseInt(prompt('donner votre age'));  
(age > 19) ? alert('vous etes majeur') :alert('vous n\'etes pas  
majeur');
```

Création d'un site web interactif

JavaScript

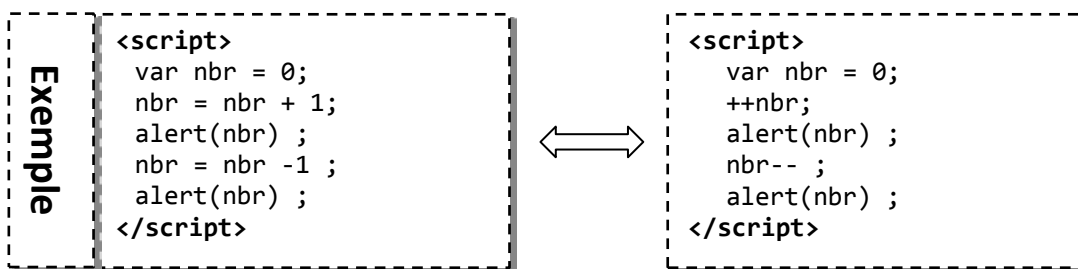
Les structures répétitives

Une boucle est une structure analogue aux structures conditionnelles sauf qu'ici il s'agit de répéter une série d'instructions. La répétition se fait jusqu'à ce qu'on dise à la boucle de s'arrêter. À chaque fois que la boucle se répète on parle d'itération (répétition).

On utilise un itérateur qui modifie la valeur de départ de la boucle à chaque nouveau passage jusqu'au moment où la condition de sortie est vérifiée. Bien souvent, on incrémentera ou décrémentera la valeur de départ à l'aide des opérateurs d'incrément et de décrémentation.

→ Les opérateurs d'incrément et de décrémentation

Les opérations d'incrément et de décrémentation vont principalement être utilisées avec les boucles en JavaScript. L'incrément permet d'ajouter une unité à un nombre au moyen d'une syntaxe courte. À l'inverse, la décrémentation permet de soustraire une unité.



→ La boucle POUR « for »

Une boucle **for** contient trois « phases » à l'intérieur du couple de parenthèses : une phase d'initialisation, une phase de test (condition de sortie) et une phase d'itération (généralement une incrément). Chaque phase est séparée des autres par un point-virgule.

Syntaxe	<pre><script> for(initialisation; condition; incrément) { instruction_1; instruction_2; instruction_3; } </script></pre>	Exemple	<pre><script> var i=0; for (i=0;i<=6;i++) { console.log ("le nbr est:"+ i); }} </script></pre>
Exemple	<pre><script> for (var prenoms = '', prenom; true;) { /*ici sans incrémentation nécessaire mais avec un point-virgule obligatoire après la condition true */ prenom = prompt('Entrez un prénom :'); if (prenom) { prenoms += prenom + ' '; } else { break; } } alert(prenoms); </script></pre>		

On peut se servir de l'incrémentation pour compter le nombre de prénoms :

Exemple

```
<script>
for (var i = 0, prenoms = '', prenom; true; i++) {
  prenom = prompt('Entrez un prénom :');
  if (prenom) { prenoms += prenom + ' '; }
  else { break; } }
alert('Il y a ' + i + ' prénoms :\n\n' + prenoms);
// Les \n servent à faire des sauts de ligne
</script>
```

→ La boucle TANT QUE «while»

L'instruction **while** permet de créer une boucle qui s'exécute tant qu'une condition de test est vérifiée. La condition est évaluée avant d'exécuter l'instruction contenue dans la boucle.

Syntaxe

```
<script>
while (condition) {
  instruction_1;
  instruction_2;
  instruction_3;
}
</script>
```

Exemple

```
<script>
var nbr = 1;
while (nbr < 10) {
  nbr++;
}
alert(nbr); // Affiche : « 10 »
</script>
```

Exemple

```
<script>
var pseudos = "", pseudo,
continuer=confirm('vous devez saisir un pseudo ');
while (continuer) {
  pseudo = prompt('Entrez un pseudo :');
  if (pseudo) {
    pseudos += pseudo + ' ';
    /* concatination de nouveau pseudo et un espace avec l'ancienne
    valeur pseudos */
  } else {
    continuer = false;
    // Aucun pseudo n'a été saisi, donc la boucle se termine
    // On peut utiliser break pour quitter la boucle
  }
}
alert(pseudos); // Affiche tous les pseudos
</script>
```

→ La boucle répéter..jusqu'à «do..while»

L'instruction **do...while** crée une boucle qui exécute une instruction jusqu'à ce qu'une condition de test ne soit plus vérifiée. La condition est testée après que l'instruction soit exécutée, le bloc d'instructions défini dans la boucle est donc exécuté au moins une fois.

Syntaxe

```
<script>
do {
  instruction_1;
  instruction_2;
  instruction_3;
}
While (condition)
</script>
```

Exemple

```
<script>
  let somme = 0;
  let nbr = 0;
  /* L'instruction let permet de déclarer une variable dont la portée est celle
  du bloc courant, éventuellement en initialisant sa valeur */
  do {
    somme += nbr;
    nbr = parseInt(prompt('Entrer un nombre: '));
  } while(nbr >= 0);
  alert('la somme est ' + somme);
</script>
```

TP

Utilisons la boucle **do-while** pour développer un jeu de devinettes simple. Le script génère un entier aléatoire entre 1 et 12. Vous devez deviner le nombre en faisant des suppositions jusqu'à ce que le nombre que vous choisissiez corresponde au nombre choisi par le script.

```
<script>
// Génération de nombre secret entre deux valeurs MIN et MAX
const MIN = 1;
const MAX = 12;
let nombresecret = Math.floor(Math.random() * (MAX - MIN + 1)) + MIN;

/* Déclaration et initialisation d'un nombre secret
La fonction Math.floor(x) renvoie le plus grand entier qui est inférieur ou égal à un
nombre x
La fonction Math.random() renvoie un nombre flottant pseudo-aléatoire compris dans l'intervalle [0, 1[ (ce qui
signifie que 0 est compris dans l'intervalle mais que 1 en est exclu) selon une distribution approximativement
uniforme sur cet intervalle. Ce nombre peut ensuite être multiplié afin de couvrir un autre intervalle */

let supposition = 0; // Enregistrement de nombre de suppositions
let indice = ' '; // Enregistrement d'indice
let nbr = 0;
do {
  // Valeur introduite par l'utilisateur
  let val = prompt('Svp entrer un nombre entre ' + MIN + ' et ' + MAX + indice);
  nbr=parseInt(val);
  // Incréméntation de nombre de supposition
  supposition++;
  // Vérification avec le nombre secret
  if (nbr > nombresecret) {
    indice = ', plus petit que ' + nbr;
  } else if (nbr < nombresecret) {
    indice = ', plus grand que ' + nbr;
  } else if(nbr == nombresecret) {
    alert('Bravo! vous avez réussi après ' + supposition + ' suppositions.');
```

```
  } while (nbr != nombresecret);
</script>
```

Création d'un site web interactif

Fiche n°10

Accès aux éléments DOM en JavaScript

Il est généralement très utile d'effectuer des contrôles sur les données saisies dans des zones de textes d'un formulaire ou sur les données sélectionnées ou cochées dans une liste, dans des boutons radios ou dans des cases à cocher.

Et pour accéder à un élément du formulaire on peut utiliser différentes manières :

Type d'accès	Méthode	Description
Accès hiérarchique	<code>document.nom_f.nom_zonedetexte</code>	
Accès aux images		Les images d'un document sont concernés par cette méthode. Il s'agit de retourner un tableau qui contient toutes les images du document courant et accéder ainsi à l'une d'entre elle via son indexe ou son nom.
Accès via l'attribut ID	<code>getElementById</code>	consiste à accéder à un élément HTML (de n'importe quel type) via son ID (qui est toujours unique).
Accès via le nom de balise	<code>getElementsByTagName</code>	Il s'agit d'une méthode largement utilisée aussi et qui consiste à retourner tous les objets de même type (même balise) en tant que tableau et en sélectionner un via son indexe (l'indexe 0 pour le premier élément, 1 pour le deuxième...).
.....	<code>getElementsByName</code>	
Accès via le nom de la classe	<code>getElementsByClassName</code>	Là encore on peut accéder à tous les objets HTML qui ont un nom de classe commun et accéder à l'un d'entre eux via son indexe.
Accès direct via l'objet THIS	<code>this</code>	C'est une méthode simple qui consiste à récupérer l'objet THIS (qui renvoie vers l'objet courant).
Accès via l'objet EVENT		Dans ce cas on peut accéder un objet (voir son enfant le plus profond) en exploitant le principe de propagation des événements
En Javascript, Il existe deux méthodes qui permettent de mettre en oeuvre <u>L'API</u> Selectors, il s'agit de	<code>querySelector</code>	Cette méthode accepte comme argument le sélecteur de l'objet (comme en CSS) et permet d'accéder au premier objet du document qui répond au sélecteur. Retourne le premier nœud Element correspondant dans le sous-arbre du nœud. Si aucun noeud correspondant n'est trouvé, NULL est renvoyé. Exemple: <pre><header> <div id="id1">1</div> <div id="id2">2</div> <div id="id3">3</div> </header> <script> alert(document.querySelector("div").getAttribute("id")); </script></pre> Après l'exécution du code on verra s'afficher le message "id1" qui correspond à la valeur de l'attribut id de la première balise <div> du document. Comme c'est le cas en CSS on peut remplacer le code Javascript précédent par l'une des lignes suivantes: <pre><script> alert(document.querySelector("#id1").getAttribute("id")); alert(document.querySelector("header>div").getAttribute("id")); </script></pre>
	<code>querySelectorAll</code>	Retourne une liste de noeuds contenant tous les noeuds Elément correspondants dans la sous-arborescence du noeud, ou une liste de noeuds vide si aucune correspondance n'est trouvée.

→ Chercher les propriétés des éléments du formulaire :

1. Les zones de textes(simples, multi-lignes ou cryptées) :

La propriété **.value** d'une zone de texte permet de récupérer la valeur d'une zone de texte.

- On peut l'affecter à une variable :
variable=document.nom_f.nom_zonedetexte.value ;
- Ou la modifier directement :
document.nom_f.nom_zonedetexte.value =valeur ;

2. Les boutons radios :

On rappelle que dans un groupe d'options le même nom est attribué à toutes les options du groupe :

- La propriété **.checked** du bouton radio permet de vérifier l'état d'une case d'option : cochée ou non.
- La propriété **.value** permet de récupérer la valeur d'une case d'option
- La propriété **.Length** permet de récupérer le nombre d'options dans un groupe.

3. Les cases à cocher

Contrairement aux boutons radios, les cases à cocher ont des noms différents pour le même groupe et plusieurs choix peuvent être sélectionnés.

- La propriété **.checked** du bouton radio permet de vérifier l'état d'une case à cocher
- La propriété **.value** permet de récupérer la valeur d'une case à cocher

4. Les listes de sélection :

Il es attribué à l'objet liste l'objet **Options** qui consiste à l'ensemble des éléments de la liste.

La syntaxe pour y accéder : document.nom_f.nom_liste.options

a. Propriétés de l'objet Options :

length : renvoie le nombre des éléments de l'objet

selectedIndex : renvoie l'indice de l'élément sélectionné de la liste.

b. Les propriétés des éléments d'un objet Options :

selected : vérifie l'état d'un élément (true s'il est sélectionné false sinon)

text : renvoie le texte d'un élément sélectionné

value : la valeur de l'élément sélectionné

c. Modification de la liste :

- Ajout d'une nouvelle option :
Variable=new Option(texte, valeur)
document.nom_f.nom_liste.options[taille] = nom_Variable ;
- Suppression d'une option :
document.nom_f.nom_liste.options[i] = null ;

JavaScript

Fonctions et évènements

→ C'est quoi une fonction?

- ✓ Une fonction est un ensemble d'instructions effectuant une tâche ou calculant une valeur. Afin d'utiliser une fonction, il est nécessaire de l'avoir auparavant définie au sein de la portée dans laquelle on souhaite l'appeler.
- ✓ Il y a les fonctions ou variables natives (déjà existantes), mais on peut aussi en créer de nouvelles, selon la structure suivante

Syntaxe	<pre><Script> // Le terme "function" est obligatoire pour déclarer une fonction function nom_fonction(arguments) { // Le code que la fonction va devoir exécuter } </Script></pre>
Exemple	<pre><script> function Multiplier_par_deux() { var result = parseInt(prompt('Donnez le nombre à multiplier par 2 :')); alert(result * 2); } Multiplier_par_deux(); // On appelle la fonction créée alert('Vous avez déjà appelé la fonction une fois !'); Multiplier_par_deux(); // Et appelle de nouveau la fonction </script></pre>

→ Les variables locales et globales

Toute variable déclarée dans une fonction n'est utilisable que dans cette même fonction. Ces variables spécifiques à une seule fonction ont un nom : les variables locales. Déclarées en dehors des fonctions, on parle de variables globales.

Exemple	<pre><script> <script> var message = 'Ici la variable globale !'; function showMsg() { var message = 'Ici la variable locale !'; alert(message); } showMsg(); // On affiche la variable locale alert(message); // Puis la variable globale </script></pre>
----------------	--

→ Les arguments

Les paramètres donnés lors de l'appel d'une fonction sont appelés les *arguments* de la fonction. Les arguments sont passés *par valeur* (*by value* en anglais). Si la fonction modifie la valeur d'un argument, ce changement ne se répercute pas en dehors de la fonction.

Exemple

```
<script>
var message = 'Ici la variable globale !';
function showMsg() {
var message = 'Ici la variable locale !';
alert(message);
}
showMsg(); // On affiche la variable locale
alert(message); // Puis la variable globale
</script>
```

--> Associer dynamiquement un évènement a un élément Appel de la fonction

Consiste a ajouter un évènement a une balise dans la page HTML alors que la page est déjà chargé ou a une balise HTML créé dynamiquement.

--> il existe trois méthodes,

Méthode1:

La première méthode qui est La plus ancienne consiste a pointer vers l'élément cible a lui associer l'évènement et lui adjoindre le nom de la fonction sans les parenthèses.

Exemple:

```
document.getElementById('mon_element').onclick=nom_de_la_fonction
```

Methode2 : addEventListener, removeEventListener

La syntaxe consiste a pointer vers l'élément cible et a appeler addEventListener en précisant l'évènement puis la fonction a exécuter.

Exemple :

```
document.getElementById('mon_element').addEventListener("click", nom_de_la_fonction, false);
```

Cette méthode contrairement a la précédente permet d' adjoindre plusieurs fonction pour le mêmes évènement.

Remarque pourquoi ajouter plusieurs fonctions sur le même évènement?

une de raison les plus connus est au chargement d'une page avec l'évènement "onload".

imaginons qu'il y ai plusieurs scripts dans une page et que ces script doivent passer par une fonction d'initialisation

Methode 3 'setAttribute'

un évènement dans une balise est considéré comme un attribut et pour cette troisième méthode nous allons utiliser 'setAttribute'.

Exemple:

```
document.getElementById('mon_element').setAttribute('onclick','nom_de lafonction('paramètre',event));
```

Création d'un site web interactif

Fiche n°11**Les méthodes et propriétés standards de l'objet Nombre, string et Array****→ L'objet Nombre**

L'objet **Number** est une enveloppe objet (*wrapper*) autour du type primitif numérique. Autrement dit, il est utilisé pour manipuler les nombres comme des objets. Pour créer un objet **Number**, on utilise le constructeur **Number()**.

Le type JavaScript **Number** utilise une représentation binaire à précision double sur 64 bits telle que décrite par le standard IEEE 754. Les implémentations plus récentes offrent un nouveau type : **BigInt** qui permet de représenter des entiers avec une précision arbitraire.

```
new Number(valeur);  
var a = new Number('123'); // a === 123 donnera false  
var b = Number('123'); // b === 123 donnera true  
a instanceof Number; // donnera true  
b instanceof Number; // donnera false
```

L'objet **Number** est principalement utilisé dans les cas de figure suivants :

- Si l'argument ne peut pas être converti en un nombre, il renverra **NaN**.
- Dans un contexte de fonction simple (quand il n'est pas utilisé comme un constructeur avec l'opérateur **new**), **Number** peut être utilisé afin d'effectuer des conversions.

→ L'objet String

Un objet **String** est utilisé afin de représenter et de manipuler une chaîne de caractères.

Il est possible de créer des chaînes de caractères comme des valeurs primitives ou comme des objets avec le constructeur **String()**

Exemple : `machaine = new String ("Un objet String") ;`

- Les chaînes de caractères peuvent être indiquées avec des simples quotes (`' '`), des doubles quotes (`" "`) ou encore par des accents graves (`` ``).
- Il existe deux façons d'accéder à un caractère dans une chaîne. La première façon consiste à utiliser la méthode **charAt()**

→ L'objet Array

L'objet global `Array` est utilisé pour créer des tableaux.

- Les tableaux sont indexés à partir de zéro: le premier élément d'un tableau a pour indice 0, et la position du dernier élément est donnée par `length` moins 1. Si on utilise un indice en dehors de cet intervalle, le résultat sera `undefined`.

Opérations sur le tableau	
Créer un tableau	<pre>let fruits = ['Apple', 'Banana']; console.log(fruits.length); // 2</pre>
Accéder (via son index) à un élément du tableau	<pre>let first = fruits[0]; // Apple let last = fruits[fruits.length - 1]; // Banana</pre>
Boucler sur un tableau	<pre>fruits.forEach(function(item, index, array) { console.log(item, index); }); // Apple 0 // Banana 1</pre>
Ajouter à la fin du tableau	<pre>let newLength = fruits.push('Orange'); // ["Apple", "Banana", "Orange"]</pre>
Supprimer le dernier élément du tableau	<pre>let last = fruits.pop(); // supprime Orange (à la fin) // ["Apple", "Banana"];</pre>
Supprimer le premier élément du tableau	<pre>let first = fruits.shift(); // supprime Apple (au début) // ["Banana"];</pre>
Ajouter au début du tableau	<pre>let newLength = fruits.unshift('Strawberry') // ajoute au début // ["Strawberry", "Banana"];</pre>
<code>Arr.length</code>	La propriété length (longueur) est un entier non-signé de 32 bits qui indique le nombre d'éléments présents dans le tableau. Elle est toujours supérieure au plus grand indice du tableau. Exemple : <pre>const clothing = ['shoes', 'shirts', 'socks', 'sweaters']; console.log(clothing.length); // expected output: 4</pre>
<code>arr.toString()</code>	Cette méthode renvoie une chaîne de caractères représentant le tableau spécifié et ses éléments. Exemple: <pre>const array1 = [1, 2, 'a', '1a']; console.log(array1.toString()); // expected output: "1,2,a,1a"</pre>

Méthode/Fonction	Description		
<code>Number.isNaN()</code>	Cette méthode permet de déterminer si la valeur passée en argument est NaN , avec un type Number . Cette version est plus robuste que la méthode de l'objet global isNaN .		
<code>numObj.toString([base])</code>	Renvoie une chaîne de caractère représentant l'objet Number. base : Paramètre optionnel. Un entier compris entre 2 et 36 qui indique la base du système numérique à utiliser pour représenter la valeur. <u>Exemple</u> : <pre> var compte = 10; console.log(compte.toString()); // affiche "10" console.log((17).toString()); // affiche "17" console.log((17.2).toString()); // affiche "17.2" var x = 6; console.log(x.toString(2)); // affiche "110" console.log((254).toString(16)); // affiche "fe" console.log((-10).toString(2)); // affiche "-1010" console.log((-0xff).toString(2)); // affiche "-11111111" </pre>		
<code>objetNumber.valueOf()</code>	Cette méthode renvoie la valeur primitive correspondant à celle représentée par l'objet Number . <u>Exemple</u> : <pre> var numObj = new Number(10); console.log(typeof numObj); // object var num = numObj.valueOf(); console.log(num); // 10 console.log(typeof num); // number </pre>		
<code>Number.parseInt(chaine [, base])</code>	Cette méthode analyse et convertit une chaîne de caractères, fournie en argument, en un entier dans la base souhaitée. **Attention : **La base par défaut n'est pas 10. Ce paramètre doit toujours être utilisé, en effet s'il n'est pas spécifié, le comportement de la fonction n'est pas garanti et peut varier d'une plate-forme à une autre. <u>Exemple</u> : <table border="1"> <tr> <td> <pre> function roughScale(x, base) { const parsed = Number.parseInt(x, base); if (Number.isNaN(parsed)) { return 0; } return parsed * 100; } </pre> </td><td> <pre> console.log(roughScale(' 0xF', 16)); // expected output: 1500 console.log(roughScale('321', 2)); // expected output: 0 </pre> </td></tr> </table>	<pre> function roughScale(x, base) { const parsed = Number.parseInt(x, base); if (Number.isNaN(parsed)) { return 0; } return parsed * 100; } </pre>	<pre> console.log(roughScale(' 0xF', 16)); // expected output: 1500 console.log(roughScale('321', 2)); // expected output: 0 </pre>
<pre> function roughScale(x, base) { const parsed = Number.parseInt(x, base); if (Number.isNaN(parsed)) { return 0; } return parsed * 100; } </pre>	<pre> console.log(roughScale(' 0xF', 16)); // expected output: 1500 console.log(roughScale('321', 2)); // expected output: 0 </pre>		
<code>Number.parseFloat(chaine)</code>	Cette méthode analyse et convertit une chaîne de caractères en un nombre flottant . Un nombre flottant obtenu à partir de l'analyse de la chaîne de caractères passée en argument. <u>Si le premier caractère de la chaîne ne peut pas être converti en un nombre, la valeur NaN sera renvoyée.</u> <u>Exemple</u> : <pre> function circumference(r) { if (Number.isNaN(Number.parseFloat(r))) { return 0; } return parseFloat(r) * 2.0 * Math.PI ; } console.log(circumference('4.567abcdefgh')); // expected output: 28.695307297889173 console.log(circumference('abcdefgh')); // expected output: 0 </pre>		
<code>eval(str)</code>	Cette fonction permet d'évaluer du code JavaScript représenté sous forme d'une chaîne de caractères. <u>Exemple</u> : <pre> console.log(eval('2 + 2')); // expected output: 4 console.log(eval(new String('2 + 2'))); </pre>		

	<pre>// expected output: 2 + 2 console.log(eval('2 + 2') === eval('4')); // expected output: true console.log(eval('2 + 2') === eval(new String('2 + 2'))); // expected output: false</pre>
str.length	La propriété length représente la longueur d'une chaîne de caractères, exprimée en nombre de points de code UTF-16. C'est une propriété accessible en lecture seule.
str.charAt(index)	Cette méthode renvoie une nouvelle chaîne contenant le caractère (à la position indiquée en argument). <u>Exemple</u> <pre>var uneChaîne = "Coucou tout le monde"; console.log(uneChaîne.charAt(0)); // affiche « C » console.log(uneChaîne.charAt(888)); // affiche ""</pre>
str.lastIndexOf(valeurRecherchée[, indiceDébut])	Cette méthode renvoie l'indice, dans la chaîne courante, de la dernière occurrence de la valeur donnée en argument. Si cette sous-chaîne n'est pas trouvée, la méthode renvoie -1. La recherche s'effectue de la fin vers le début de la chaîne, à partir de indiceDébut. <u>Exemple :</u> <pre>'canal'.lastIndexOf('a'); // renvoie 3 'canal'.lastIndexOf('a', 2); // renvoie 1 'canal'.lastIndexOf('a', 0); // renvoie -1 'canal'.lastIndexOf('x'); // renvoie -1 'canal'.lastIndexOf('c', -5); // renvoie 0 'canal'.lastIndexOf('c', 0); // renvoie 0 'canal'.lastIndexOf(""); // renvoie 5 'canal'.lastIndexOf("", 2); // renvoie 2</pre> Note : 'abab'.lastIndexOf('ab', 2) renvoie 2 et pas 0 car l'argument indiceDébut ne limite que le début de la correspondance recherchée (qui est 'ab')
str.indexOf(valeurRecherchée) str.indexOf(valeurRecherchée, indiceDébut)	Cette méthode renvoie l'indice de la première occurrence de la valeur cherchée au sein de la chaîne courante (à partir de indiceDébut). Elle renvoie -1 si la valeur cherchée n'est pas trouvée. <u>Exemple :</u> <pre>"Blue Whale".indexOf("Blue"); // retourne 0 "Blue Whale".indexOf("Blute"); // retourne -1 "Blue Whale".indexOf("Whale", 0); // retourne 5 "Blue Whale".indexOf("Whale", 5); // retourne 5 "Blue Whale".indexOf(""); // retourne 0 "Blue Whale".indexOf("", 9); // retourne 9 "Blue Whale".indexOf("", 10); // retourne 10 "Blue Whale".indexOf("", 11); // retourne 10</pre>
chn.substr(début[, longueur])	Cette méthode retourne la partie d'une chaîne de caractères comprise entre l'indice de départ et un certain nombre de caractères après celui-ci. <u>Exemple</u> <pre>var uneChaîne = 'Mozilla'; console.log(uneChaîne.substr(0, 1)); // 'M' console.log(uneChaîne.substr(1, 0)); // '' console.log(uneChaîne.substr(-1, 1)); // 'a' console.log(uneChaîne.substr(1, -1)); // '' console.log(uneChaîne.substr(-3)); // 'lla' console.log(uneChaîne.substr(1)); // 'ozilla' console.log(uneChaîne.substr(-20, 2)); // 'Mo' console.log(uneChaîne.substr(20, 2)); // ''</pre>
str.substring(indiceA[, indiceB])	Cette méthode retourne une sous-chaîne de la chaîne courante, entre un indice de début et un indice de fin. <u>Exemple</u> <pre>var uneChaîne = "Mozilla"; // Affiche "Moz" console.log(uneChaîne.substring(0,3)); console.log(uneChaîne.substring(3,0)); // Affiche "lla" console.log(uneChaîne.substring(4,7)); console.log(uneChaîne.substring(4)); console.log(uneChaîne.substring(7,4)); // Affiche "Mozill" console.log(uneChaîne.substring(0,6)); // Affiche "Mozilla" console.log(uneChaîne.substring(0,7)); console.log(uneChaîne.substring(0,10));</pre>

str.toUpperCase()	Cette méthode retourne la valeur de la chaîne courante, convertie en majuscules. Exemple : <pre>console.log("alphabet".toUpperCase()); // "ALPHABET"</pre>
str.toLowerCase()	Cette méthode retourne la chaîne de caractères courante en minuscules. Exemple <pre>console.log("ALPHABET".toLowerCase()); // "alphabet"</pre>
str.concat(string2[, string3, ..., stringN])	Cette méthode combine le texte de plusieurs chaînes avec la chaîne appelante et renvoie la nouvelle chaîne ainsi formée. Exemple <pre>var coucou = "Bonjour "; console.log(coucou.concat("Tristan, ", " bonne journée.")); /* Bonjour Tristan, bonne journée. */ console.log(concat(4, 5)); // "45"</pre>
str.trim()	Cette méthode permet de retirer les blancs en début et fin de chaîne. Les blancs considérés sont les caractères d'espacement (espace, tabulation, espace insécable, etc.) ainsi que les caractères de fin de ligne (LF, CR, etc.). Exemple : <pre>const greeting = ' Hello world! '; console.log(greeting); // expected output: " Hello world! " console.log(greeting.trim()); // expected output: "Hello world!"</pre>
str.split([separator[, limit]])	Cette méthode divise une chaîne de caractères en une liste ordonnée de sous-chaînes, place ces sous-chaînes dans un tableau et retourne le tableau. La division est effectuée en recherchant un motif ; où le motif est fourni comme premier paramètre dans l'appel de la méthode. Exemple : <pre>const str = 'The quick brown fox jumps over the lazy dog.'; const words = str.split(' '); console.log(words[3]); // expected output: "fox" const chars = str.split(""); console.log(chars[8]); // expected output: "k" const strCopy = str.split(); console.log(strCopy); // expected output: Array ["The quick brown fox jumps over the lazy dog."]</pre>

Création d'un site web interactif

Fiche n°12

JavaScript La gestion du temps

→ C'est quoi La gestion du temps?

- ✓ Il s'agit en fait, en Javascript, du nombre de millisecondes écoulées depuis le 1er janvier 1970 à minuit. Cette manière d'enregistrer la date est très fortement inspirée du système d'horodatage utilisé par les systèmes Unix. La seule différence entre le système Unix et le système du Javascript, c'est que ce dernier stocke le nombre de millisecondes, tandis que le premier stocke le nombre de secondes. Dans les deux cas, ce nombre s'appelle un **timestamp**.
- ✓ Ce nombre ne nous sert vraiment qu'à peu de choses à nous, développeurs, car nous allons utiliser l'objet **Date** qui va s'occuper de faire tous les calculs nécessaires pour obtenir la date ou l'heure à partir de n'importe quel **timestamp**.

→ L'objet Date ?

L'objet **Date** nous fournit un grand nombre de méthodes pour lire ou écrire une date.

- ✓ Le constructeur : Commençons par le constructeur ! Ce dernier prend en paramètre de nombreux arguments et s'utilise de différentes manières. Voici les quatre manières de l'utiliser :

<code>new Date();</code>	Instancie un objet Date dont la date est fixée à celle de l'instant même de l'instanciation.
<code>new Date(timestamp);</code>	Permet de spécifier le timestamp à utiliser pour notre instanciation.
<code>new Date(dateString);</code>	Prend une chaîne de caractères qui doit être interprétable par la méthode parse() .
<code>new Date(année, mois, jour [, heure, minutes, secondes, millisecondes]);</code>	Permet de spécifier manuellement chaque composant qui constitue une date.

- ✓ Les méthodes de l'objet Date : Étant donné que l'objet Date ne possède aucune propriété standard, nous allons directement nous intéresser à ses méthodes qui sont très nombreuses

<code>getFullYear();</code>	Renvoie l'année sur 4 chiffres ;
<code>getMonth();</code>	Renvoie le mois (0 à 11) ;
<code>getDate();</code>	Renvoie le jour du mois (1 à 31) ;
<code>getDay();</code>	Renvoie le jour de la semaine (0 à 6, la semaine commence le dimanche) ;
<code>getHours();</code>	Renvoie l'heure (0 à 23) ;
<code>getMinutes();</code>	Renvoie les minutes (0 à 59) ;
<code>getSeconds();</code>	Renvoie les secondes (0 à 59) ;
<code>getMilliseconds();</code>	Renvoie les millisecondes (0 à 999).
<code>getTime();</code>	Renvoie le timestamp de la date de votre objet ;
<code>setTime();</code>	Permet de modifier la date de votre objet en lui passant en unique paramètre un timestamp.

Exemple 1

```
<script>
//Afficher l'heure actuelle du système
var d = new Date();
var heure = d.getHours() + ":" + d.getMinutes() + ":" + d.getSeconds();
alert(heure);
</script>
```

Exemple 2

```
<script>
// Exemple d'affichage de gestion d'un timestamp
var myDate = new Date('Sat, 04 May 1991 20:00:00 GMT+02:00');
//Creation d'un objet date avec timestamp comme argument
alert(myDate.getMonth()); // Affiche : 4
// Faites attention les mois commencent par 0
alert(myDate.getDate()); // Affiche le mois : 4
alert(myDate.getDay()); // Affiche le jour : 6
</script>
```

→ Tp : Création widget horloge avec javascript

TP (Horloge numérique)

```
<html>
<head>
<script>
function rafraichir(){
    var t = 1000; // rafraîchissement en millisecondes
    setTimeout('Affichehorloge()',t)
    //setTimeout: déclenche des actions après un intervalle de temps donné
    //On doit déclencher Affichehorloge() de nouveau après 1000 ms
    //1 Millisecondes = 0.001 Secondes
}

function Affichehorloge() {
    var date = new Date()
    //Création de l'objet date
    var h = date.getHours();
    //Extraire l'heure de l'objet date
    var m = date.getMinutes();
    //Extraire les minutes de l'objet date
    var s = date.getSeconds();
    //Extraire les secondes de l'objet date
    if( h < 10 ){ h = '0' + h; }
    if( m < 10 ){ m = '0' + m; }
    if( s < 10 ){ s = '0' + s; }
    var time = h + ':' + m + ':' + s
    document.getElementById('horloge').innerHTML = time;
    //mise à jour du span horloge avec innerHTML
    rafraichir();
    //appel de la fonction pour le rafraîchissement des variables de nouveau
}
</script>
</head>
<body onload=Affichehorloge();>
    <span id='horloge' style="background-color:black ;color:silver;font-size:100px;"></span>
</body>
</html>
```

innerHTML est une propriété de tout élément HTML qui désigne le contenu qui se trouve entre la balise entrante et la balise fermante. On l'utilise pour lire pour insérer dynamiquement un contenu.

Soit l'élément suivant:

```
<p id="parag">
un texte. </p>
```

On peut obtenir le contenu du paragraphe avec ce code:

```
var x = document.getElementById("parag");
alert(x.innerHTML);
```



On peut remplacer dynamiquement le contenu du paragraphe :

```
var x =  
document.getElementById("parag");  
x.innerHTML = "un autre texte";
```

Module 1

Création d'un site web dynamique

Fiche n°13

PHP

I. Introduction :



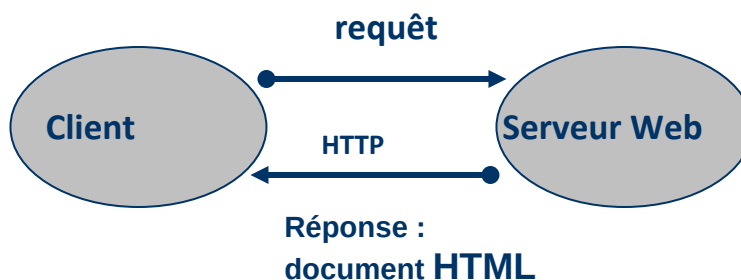
Un site web dynamique est un site dont les pages peuvent être générées en fonction d'une demande d'un utilisateur, donc le traitement est du côté serveur.

→ Les intérêts du traitement côté serveur

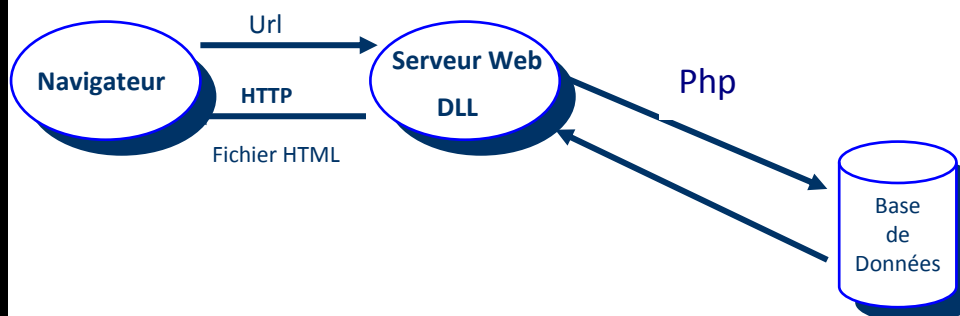
- réduction du temps de téléchargement puisque le client ne reçoit qu'une simple page HTML (=> diminution du trafic réseau).
- absence de problème de compatibilité des navigateurs.
- offrir au client des données qui sont dans une base.
- le code ne peut être vu par le client.

→ Principe de fonctionnement :

Consultation d'une page Web statique



Consultation d'une page Web dynamique



Donc Php s'exécute côté serveur. Donc il n'y a aucune trace du code Php lorsqu'on consulte le code source de la page dans le navigateur du client.

II. Le langage PHP :

II.1 Présentation :

PHP (abréviation de Personal Home Page Hypertext Preprocessor), est un langage de scripts qui s'intègre aux pages Html et qui permet de réaliser des pages dynamiques.

- Il s'exécute sur le serveur et permet d'accéder facilement aux bases de données.
- C'est un produit "Open Source" c'est-à-dire que le code est accessible à tout développeur.
- Il est gratuit.
- Combiné au système d'exploitation Linux, au serveur Apache et à la base de données MySQL (eux-mêmes gratuits), il permet de créer des sites Web à des coûts très réduits.
- Simple à utiliser.

II.2 Environnement de développement :

Pour le développement d'un site Web dynamique, il faut installer en local un serveur Web (on peut alors programmer sans avoir besoin de se connecter à Internet).

Donc il faut installer :

un serveur Web (exemple : Apache)

Serveur de BD (exemple : Mysql, Oracle,...)

Mais il existe un utilitaire nommé **EasyPhp** qui installe automatiquement Apache, Php, Mysql et aussi phpMyAdmin.

→ Utilisation d'une première page :

Enregistrement de la page_	Doit se faire toujours sous le répertoire www La page est généralement enregistrée avec l'extension php ou php3 ou php4 sinon le serveur http(Apache) ne peut pas comprendre les balises < ? et ?>
Affichage du résultat	- il faut s'assurer que le serveur Apache est démarré Et pour accéder à la page Php, il existe 2 façons : soit par « Web local » soit en ouvrant le navigateur et saisir http://localhost ou bien http://127.0.0.1

Ce qui se passe entre le client et le serveur

Lorsqu'un internaute demande page depuis son navigateur, voici ce qui s'y passe :

- l'internaute demande une page (exemple : <http://www.Bacinfo.tn/page.php>)
- elle arrive au serveur Web
- elle est analysée et exécutée par l'interpréteur PHP
- elle est transformée en langage HTML
- elle est renvoyée sur le navigateur du poste client via Internet. L'internaute ne peut absolument pas voir le code source de la page PHP.

II.3 Syntaxe de base du langage Php :

Le script s'intègre directement au code HTML et commence par `<? (ou <?php)` et se termine par `?>`.
Les commentaires

//	sur une même ligne, tout ce qui suit les signes // n'est pas étudié par l'analyseur PHP
/* ... */	Sur plusieurs lignes, tout ce qui est compris entre /* et */ n'est pas étudié par l'analyseur. Ceci permet de déterminer des blocs de commentaires.

II.4 Les entrées/sorties:

- Les saisies de données (entrée) font être réalisées par l'intermédiaire des formulaires.

On utilise soit :

`$_GET['nomobjet']` (dans le cas ou METHOD de la balise FORM est GET)

`$_POST['nomobjet']` (dans le cas ou METHOD de la balise FORM est POST)

- Les édition des résultats (sorties) vont s'inscrire dans la page par les fonctions

`echo ("message");`

`print ("message");`

II.5 Les structures de données :

a) Les constantes :

Pour utiliser une constante, il faut utiliser la fonction `define()`.

Exemple : `echo (" <HR> " . font . ecole);` // le signe . signifie que les variables seront concaténées.

b) Les variables :

Déclaration d'une variable	En php les variables sont représentées par le signe dollar \$ suivi par le nom de la variable. Une variable est automatiquement déclarée dès que vous lui affectez une valeur.
Conversions de types	Settype

Important : Il est à noter que malgré ces déclarations, Php ne contrôle pas le contenu des variables c'est-à-dire qu'à posteriori dans le programme, si on place du texte dans une variable numérique, Php convertira tout seul le type de la variable en "string".

c) Fonctions de manipulation de variable :

Fonctions	Commentaires	Exemples
Gettype (\$nom_var)	détermine le type de données de la variable ("integer", "double", "string", "array", "object", "class" ou "unknown type")	<code>\$x = 2.3;</code> <code>echo (gettype(\$x));</code> // double
Settype(\$nom_var,"type")	définit explicitement le type de variable	<code>\$x = 2.3;</code> <code>settype (\$x, "integer");</code> <code>echo \$x; // \$x=2</code>
Isset(\$nom_var)	sert à savoir si une variable possède une valeur (true ou false)	<code>\$x = 2.3;</code> <code>\$val = isset (\$x) ;</code> // true

Unset(\$nom_var)	détruit une variable	unset (\$x); // isset(\$x) donne false
------------------	----------------------	--

d) Les opérateurs :

II.6 Les structures de contrôle conditionnelles :

<pre> If (condition) { Expression vrai } Else {expression faux} Switch (expression) { case R1 : instructions1 Break ; case R2 : instructions2 Break ; ... Default : instructions_N }</pre>
--

II.7 Les structures itératives:

Pour...	Exemple:somme des 10 premiers nombres entiers
<pre> for (expression_initiale ; condition_de_sortie ; expression de progression) { instructions; }</pre>	<pre> for (\$i=1 ; \$i <= 10 ; \$i++) // pour i variant de 1 à 10 // (inclus) par pas de 1 { \$somme = \$somme + \$i; }</pre>

Dans une page HTML, si la boucle est entrecoupée de code HTML, on déterminera la fin de la boucle par l'instruction `<? endfor ?>`

Exemple : `<? for ($1 = 1 ; $i < 4 ; $i++) ?>`
`<H3> Hello ! </H3>`
`<? endfor ?>`

Tantque...	Exemple : factorielle 10
<pre> while (condition) { instructions; }</pre>	<pre> \$n = 10 ; \$somme = 1 ; while (\$n > 0) // tant que n > 0 (inclus) par pas de 1 { \$somme = \$somme * \$n; \$n-- ; }</pre>

Faire ... Tant que...	Exemple : liste des élèves de 4ème année
-----------------------	--

<pre>do { instructions; } while (condition);</pre>	<pre>\$texte=""; // initialisation de la chaîne \$n=0; \$option_4a = array ("agrotic", "forêt", "oeno", "fin"); // création tableau des options de 4ème année do { \$texte = \$texte . \$option_4a [\$n++] . "\n"; //concatène les options et ajoute un retour ligne \n } while (\$option_4a[\$n] != "fin"); // tant que différent du mot "fin" echo (\$texte);</pre>
--	---

II.7 Les types de données structurées en Php :

II.7.1 Les tableaux:

Avec PHP, il existe plusieurs façons de déclarer des tableaux unidimensionnels ou multidimensionnels. Il est possible de stocker des éléments de types différents dans un même tableau.

✓ Déclaration d'un tableau à une dimension :

<pre>\$dept_enita = array ("des", "dpa", "dbe"); ou \$dept_enita[] = "des"; \$dept_enita[] = "dpa"; \$dept_enita[] = "dbe"; ou \$dept_enita[0] = "des"; \$dept_enita[1] = "dpa"; \$dept_enita[2] = "dbe";</pre>	Tableau de données. Dans tous les cas, on obtiendra : dept_enita [0] = "des" ; dept_enita [1] = "dpa" ; dept_enita [2] = "dbe" ;
---	---

Remarque : si on ajoute au code précédent la ligne \$dept_enita[10]="formco", alors la valeur de l'indice de \$dept_enita[] sera 11.

a) L'initialisation :

Il existe plusieurs façons d'insérer des éléments dans un tableau :

	Exemple :
On peut tout simplement leur affecter des valeurs, de la même façon que pour les autres variables.	\$tab[0]="P" ou même \$tab[] = "P"

On peut initialiser les éléments d'un tableau en utilisant la fonction array().	<code>\$tab=array("P",2);</code> <code>\$vente=array("lundi">7,"mardi">5,"jeudi">9);</code>
---	---

b) Le parcours d'un tableau :

Tout tableau possède un pointeur interne qui conserve l'indice et la valeur de l'élément actif. Il est possible de déterminer la valeur de l'élément actif à l'aide de la fonction `current()` et l'indice de l'élément actif à l'aide de la fonction `key()`.

Les deux fonctions **`each()` et `list()`** peuvent être utilisées conjointement afin de parcourir un tableau :

Exemple :

<pre>While (list(\$key,\$value) = each(\$vente_hebdo)) { Echo "
 Les ventes du \$key : \$value unites
"; }</pre>

Il existe une série de fonctions permettant de se déplacer dans un tableau.

Le tri d'un tableau :	
Pour trier un tableau dans l'ordre croissant	<code>sort()</code>
Pour trier un tableau dans l'ordre décroissant	<code>rsort()</code>
Pour trier un tableau par indice à l'aide des fonctions	<code>ksort()</code> et <code>krsort()</code>

II.7.2 Les chaînes de caractères :

Une chaîne de caractères est un ensemble de caractères entre guillemets simples ou doubles.

Les fonctions de chaînes de caractères :	
Fonction	Commentaire
String substr (string source,int début,int [taille]	
String trim(string str)	Retourne la chaîne nettoyée
Int Strlen(string str)	Retourne la longueur de la chaîne
strpos(<i>str,find,début</i>)	Retourne la 1 ^{ère} occurrence d'une chaîne dans une autre
strrev(<i>string</i>)	Renverse une chaîne donnée
strtoupper(<i>string</i>)	Conversion de toute la chaîne donnée en majuscule
strtolower(<i>string</i>)	Conversion de toute la chaîne donnée en miniscule
strcmp(<i>string1,string2</i>)	Compare deux chaînes données
String str_replace(modèle,remplacement,chaîne)	Remplace toutes les occurrences de modèle dans chaîne par remplacement
String implode(string separator,array tableau)	Retourne une chaîne constituée de tous les éléments du tableau,pris dans l'ordre, transformés en chaîne,et séparés par séparateur
Array Explode(string separator, string string)	Retourne un tableau contenant les éléments de la chaîne string, séparés par separator

II.8 Les fonctions en Php :

- une fonction réalise une succession d'instructions et renvoie une et une seule valeur issue d'un calcul (instruction return)
- une procédure réalise une succession d'instructions mais ne renvoie pas de valeur en retour.

La déclaration d'une fonction

```
function Nom_De_La_Fonction(argument1, argument2, ...) {  
    liste d'instructions...  
    [return $valeur_de_retour ;]}
```

- ♦ La valeur de retour est optionnelle
- ♦ Les arguments sont facultatifs, mais s'il n'y a pas d'arguments, les parenthèses doivent rester présentes
- ♦ Si la valeur initiale peut être modifiée dans la fonction on doit précéder le nom de l'argument par le symbole &.

Appel d'une fonction

```
Nom_De_La_Fonction(argument1, argument2,...);
```

II.9 Utiliser MySql avec Php :

L'utilisation de MySql avec Php s'effectue en 4 étapes :

1. Définition de la connexion
2. sélection de la base
3. exécution d'une requête SQL
4. Exploitation d'une requête SQL

Définition de la connexion	int mysql_connect (string hostname, string username, string password)
Sélection de la base de données	int mysql_select_db (string nom_bd,[int lien_identificateur])
Exécution de la requête	int mysql_query (string requete [, int lien_identificateur]) (envoie au serveur MySql une instruction SQL à executer)
Exploitation d'une requête SQL	array mysql_fetch_array (int result, int result_type)
	int mysql_affected_rows()
	Int mysql_insert_id(int lien_identificateur)
	Mysql_insert_id()

Exemple insertion	Exemple Affichage
--------------------------	--------------------------

<pre> <html> <head> <title>Liens</title> </head> <body> <table border="1" cellpadding="0" cellspacing="0"> <tr> <th>Nom du site</th> <th>URL</th> </tr> <?php // Déclaration des paramètres de connexion \$host = la_machine; // Généralement la machine est localhost // c'est-a-dire la machine sur laquelle le script est hébergé \$user = votre_login; \$bdd = Nom_de_la_base_de_donnees;</pre>	<pre> \$passwd = Mot_de_passe; // Connexion au serveur mysql_connect(\$host, \$user,\$passwd) or die("erreur de connexion au serveur"); mysql_select_db(\$bdd) or die("erreur de connexion a la base de donnees"); // Creation et envoi de la requete \$query = "SELECT nom,url FROM sites ORDER BY nom"; \$result = mysql_query(\$query); // Recuperation des resultats while(\$row = mysql_fetch_row(\$result)){ \$Nom = \$row[0]; \$url = \$row[1]; echo "<tr>\n <td> \$Nom </td>\n <td>\$url</td>\n </tr>\n"; } // Déconnexion de la base de données mysql_close(); ?> </tr> </table> </body> </html></pre>
--	--

Module 2

Bases de données

Fiche n°1

Notion de Bases de données(Rappel)

Une donnée : est une information qui peut effectuer des traitements	Une information : c'est une donnée regroupée avec d'autres liées à un contexte donnée.	Entité : c'est la représentation d'un objet du monde réel ayant une existence propre. (Exp : Etudiant, Livre, classe...)
---	--	--

Propriété (attribut) : représente les caractéristiques les propriétés de l'entité.	Association : c'est un outil permettant de représenter les liens entre les entités.	Base de Données : est un ensemble de données non redondantes, structurées et persistantes (non volatiles) relatives à un ou plusieurs domaines du monde réel.
Intérêts de l'utilisation d'une BD : -Centralisation -Indépendance entre données et programme -Intégrité de données -Partage de données (concurrence d'accès)	Intégrité de données : Ensemble de règles appelées contraintes d'intégrité qui décrivent les propriétés qui doivent être vérifiées par les données pour être enregistrées. Elle permet d'assurer la cohérence des données.	Modèles de BD : -Hiérarchique -Réseau - Modèle Relationnel : le principe consiste à représenter aussi bien les objets que les liens à l'aide d'une structure appelée table .
Système de Gestion de Base de Données(SGBD) : c'est un logiciel qui permet de créer, de gérer et d'interroger efficacement une BD. (Pour dialoguer avec le SGBD il faut utiliser le langage SQL (Structured Query Language))	Les fonctions de bases d'un SGBD : -Définition des données -Manipulation des données -Intégrité des données -la gestion des accès concurrents -la confidentialité -la sécurité de fonctionnement	Table : une table est un ensemble de données relatives à un même sujet(ou entité) et structurées sous forme de tableau. Colonne (ou champ ou propriété) : Dans une table une colonne est une propriété élémentaire de l'objet décrit par cette table. Ligne (n-uplet ou enregistrement) : Une ligne correspond à un objet représenté par la table.
Clé primaire (identifiant=primary key) : c'est un identificateur unique pour la table qui correspond à une colonne ou un ensemble de colonnes.	Clé étrangère =référence =contrainte d'intégrité référentielle = foreign key Un lien entre deux tables A et B est représenté par l'ajout dans la table B d'une nouvelle colonne correspondant à la clé primaire de la table A. cette nouvelle colonne est appelée Clé étrangère .	Contraintes d'intégrité : Ensemble de règles appliquées à une colonne ou une table et qui doit être toujours vérifiée. Les principales contraintes d'intégrité sont : Contraintes de domaines : Ce sont des contraintes appliquées sur les colonnes. Elles permettent de fixer caractère obligatoire ou non d'une colonne et les règles de validité des valeurs qui peuvent être prise par cette colonne. Contraintes d'intégrité de table : Elles permettent d'assurer que chaque table a une clé primaire. Contrainte d'intégrité référentielle : Elles permettent de s'assurer que les valeurs introduites dans une colonne se figurent dans une autre comme clé primaire.

****Exemples pour les relations ****

Relation un à plusieurs

Un élève appartient à une seule classe. Dans une classe on trouve plusieurs élèves

ELEVE ∞

1 CLASSE

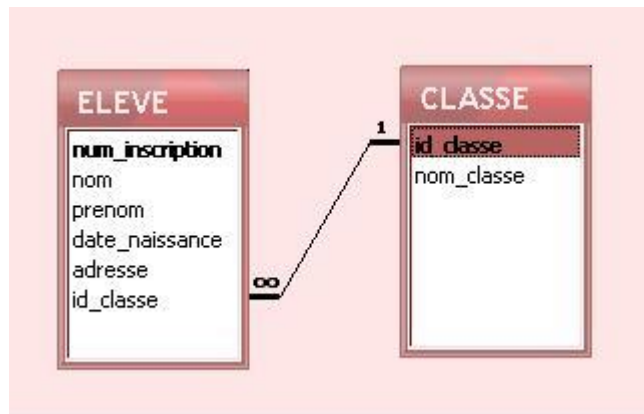
Donc CLASSE table mère et ELEVE table fille. Comme CLASSE est mère alors sa clé primaire migre à la table fille qui est ELEVE et sera une clé étrangère

Description textuelle :

ELEVE (num_inscription, nom, prenom, date_naissance, adresse, id_classe#)

CLASSE(id_classe, nom_classe)

Description graphique :



Relation plusieurs à plusieurs

Un athlète peut participer à plusieurs courses. Dans une course on trouve plusieurs athlètes.

Dans ce cas la relation participer se transforme en un table identifié par deux clés primaires qui sont les clés primaires de la table athlète et la table Courses(les deux clés seront primaires et étrangères au même temps dans la nouvelle table)

Description textuelle :

athlete (anum,anom,aage)

participation (cnum#,anum #)

course (cnum,cnom)

Description graphique :



Base de données relationnelle

Exemple : la table élève

N°	Nom	Prénom	Age
1	Benzarti	Walid	18
2	Soussi	Alia	22
3	Tounsi	Salah	19

TABLES

Table mère

Table fille

LIENS ENTRE TABLES (1, ∞)

CONTRAINTES

1- d'intégrité de tables

2- d'intégrité référentielles

3- de domaines

Exemple

Table Elève
(Table fille)

NuméroElève	Nom	Prénom	Age	NuméroClasse
1	Tounsi	Salah	18	C1
2	Benzarti	Leila	22	C1
3	Gabsi	Ahmed	19	C2

Table Classe (Table mère)

NuméroClasse	Niveau	NombreElève
C1	3eme info	24
C2	3eme lettres	19
C3	4eme info	20

N°	Numérique	2	Oui	>0 et <=30
Nom	Texte	25	Oui	
Prénom	Texte	25	Oui	
Age	Numérique	3	Oui	>0

Colonnes

3	Tounsi	Salah	19
---	--------	-------	----

(Db2, Microsoft SQL Server, MySQL, PostgreSQL, Oracle Database, SQLite...)

Création et modification de la structure d'une base de données

Le mode commande consiste à créer les différentes structures de la base de données à l'aide de commandes du langage **SQL (Structured Query Language)**

SQL est un langage informatique utilisé pour exploiter des bases de données. Il permet de façon générale la définition, la manipulation et le contrôle de sécurité de données.

Dans la pratique, le langage **SQL** est utilisé pour créer des tables, ajouter des enregistrements sous forme de lignes, interroger une base de données, la mettre à jour, ou encore gérer les droits d'utilisateurs de cette base de données.

Ce langage est composé de trois familles de commandes:

1. Langage de définition de données (LDD)
 - création de tables : **CREATE TABLE**
 - modification de tables: **ALTER TABLE**
 - suppression de tables: **DROP TABLE**
2. Langage de manipulation de données (LMD /DML)
 - insertion de lignes: **INSERT**
 - mise à jour des lignes: **UPDATE**
 - suppression de ligne: **DELETE**
 - Langage de requêtes : **SELECT FROM..... WHERE.....**
3. Langage de contrôle de données: GRANT, REVOKE

Soit la description textuelle de la base de données d'une agence de location de voitures:

Client (NCIN_Cli, Nom_Cli, Prenom_Cli, Adresse_Cli, Ville)

Vehicule (Immat_vehicule, Marque, Model, Annee_Acq)

Location (Immat_Vehicule#, NCIN#, Date_Loc, Duree_Loc, Cout_Loc)

I. Langage de définition de données (LDD)

I.1 Création de la BD:

Les étapes à suivre:

1. Taper la commande "**CREATE DATABASE Gestion_**

2. Pour utiliser la base déjà créée, taper la commande "**USE Gestion_Location;**"

Remarque:

Les commandes "**CREATE DATABASE <nom_de_la_base>**" et "**USE <nom_BD>**" sont des commandes SQL propres au SGBD MySQL et non des commandes normalisées comme celle qu'on va les voir à la suite de ce cours.

Il s'agit de même de trois commandes:

- "**SHOW DATABASES**" pour afficher les noms des bases de données existantes
- "**SHOW TABLES**" pour afficher les tables d'une base de données déjà utilisée
- "**DESCRIBE <nom_table>**" affiche pour la table "nom_table" ses colonnes ainsi que leurs types et contraintes de domaine s'il existe.

I.2 Création d'une table

La commande SQL qui permet de créer une table en donnant son nom, ses attributs et ses contraintes est:

```
CREATE TABLE nom_table (  
  Nom_Colonne1 type_colonne1 [[CONSTRAINT] contrainte-col]*  
  ...  
  [CONSTRAINT] contrainte-table);
```

Légende:

[]:option *: applicable autant de fois que souhaité **mots en capitale** : mots-clé.

Noter Bien:

❖ **Contrainte-col : contrainte sur une colonne**

- **NOT NULL** : obligatoire
- **UNIQUE** : pas deux lignes avec la même valeur
- **PRIMARY KEY** : identifiant de la table (UNIQUE + NOT NULL)
- **REFERENCES** nom_table (nom-col)
- **CHECK (condition)**: Condition que chaque ligne de la table doit vérifier
- **AUTO_INCREMENT** : s'applique pour une colonne de type entier (identique au type Numéro_Auto en Ms. Access)

❖ **Contrainte-table : contraintes sur une table**

- **PRIMARY KEY** (nom-col1, [nom-col2,...])
- **FOREIGN KEY** (nom-col)* **REFERENCES** nom_table [(nom-col)*]
- **CHECK** (condition)

❖ **Les principaux types de données sous MySQL**

✓ **Les champs numériques :**

TINYINT	Entier très petit compris entre -128 et 127 Si l'option UNSIGNED est utilisée, ce nombre sera compris entre 0 et 255
INT	Entier standard compris entre -2 147 483 648 et 2 147 483 647. Si l'option UNSIGNED est utilisée, ce nombre sera compris entre 0 et 4 294 967 295
NUMERIC (n)	Décimal de double précision
DECIMAL (entier, décimal)	Réel, définissez la longueur de chacune des deux parties.

✓ **Les champs chaînes de caractères :**

CHAR (n)	Chaîne de n caractère, taille fixe
VARCHAR(M)	Chaîne de caractères variable. M peut être compris entre 1 et 255.(avec 255 caractères maximum)
BLOB, TEXT	Zone de texte standard Objet d'une longueur maximale de 65535 caractères, TEXT aura un contenu de type ASCII (casse insensible) et BLOB aura un contenu de type binaire (casse sensible).(avec 65 535 caractères maximum)

✓ **Les champs de type date et heure :**

DATE	Date (ex: 2021-11-04)
TIME	Heure (ex: 23:44:05)
DATETIME	Date et heure (ex: 2021-11-04 23:44:05)
YEAR	Année (ex: 2021)

Remarque:

❶ Qu'est ce qui se passe quand on détruit une clé primaire qui est référencée par un tuple (foreign key) d'une autre table?

Pour supprimer aussi la ligne de la table fille qui correspond à la clé primaire de la table mère, il faut utiliser l'action "**ON DELETE CASCADE**" lors de la définition de clé étrangère.

❷ Lors de la création des tables en SQL, on doit toujours commencer par les tables mères.

Activité 10

Créer les trois tables **Client**, **Vehicule** et **location**

Les commandes SQL permettant la création de 3 tables sont:

❖ **Table Client**

```
CREATE TABLE Client(  
  NCIN_Cli VARCHAR(8) PRIMARY KEY,  
  Nom_Cli VARCHAR(20) NOT NULL,  
  Prenom_Cli VARCHAR(20) NOT NULL,  
  Adresse_Cli VARCHAR(50),  
  Ville VARCHAR(20));
```

❖ **Table Véhicule**

```
CREATE TABLE Vehicule(  
  immat_Vehicule VARCHAR(10) primary key,  
  marque VARCHAR(20) NOT NULL CHECK(marque  
  in('Peugeot','Renault','Fiat','Opel','Volkswagen','Citroën')),  
  model VARCHAR(20) NOT NULL,  
  date_acq YEAR NOT NULL);
```

❖ **Table Location**

```
CREATE TABLE Location(  
  immat_Vehicule VARCHAR(10) REFERENCES vehicule(immat_Vehicule )ON DELETE  
  CASCADE,  
  NCIN_Cli VARCHAR(8) REFERENCES Client(NCIN_Cli )ON DELETE CASCADE,  
  date_loc TIMESTAMP DEFAULT CURRENT_TIMESTAMP C HECK(date_loc<=NOW()),  
  duree_loc INT(3) CHECK(duree_loc>0),  
  cout_loc DECIMAL(8,3) CHECK(cout_loc>0),  
  CONSTRAINT PK Location PRIMARY KEY(immat_Vehicule,NCIN_Cli));
```

NB: Pour attribuer la date d'aujourd'hui comme valeur par défaut, il faut utiliser le type "timestamp" sinon ça ne marche pas avec le type date.

Illustrations graphiques de la création de différentes tables:

Table Client

```
MySQL Command Line Client
mysql> describe client;
```

Field	Type	Null	Key	Default	Extra
NCIN_Cli	varchar(8)	NO	PR		
nom_Cli	varchar(20)	NO			
Prenom_Cli	varchar(20)	NO			
adresse_Cli	varchar(60)	YES		NULL	
Ville	varchar(20)	YES		NULL	

5 rows in set (0.02 sec)

❖ Table Vehicle

```
MySQL Command Line Client
mysql> create table Vehicule
-> (immat_Vehicule varchar(10) primary key,
-> marque varchar(20) NOT NULL CHECK(marque in ('Peugeot','Renault','Citroen','Volvo','Nissan','Ford','Honda','Toyota')),
-> model varchar(20) NOT NULL,
-> date_acq year NOT NULL);
Query OK, 0 rows affected (0.48 sec)

mysql> describe vehicule;
```

Field	Type	Null	Key	Default	Extra
immat_Vehicule	varchar(10)	NO	PRI		
marque	varchar(20)	NO			
model	varchar(20)	NO			
date_acq	year(4)	NO			

4 rows in set (0.02 sec)

❖ Table Location

```
MySQL Command Line Client
mysql>
mysql> create table Location(
-> immat_Vehicule varchar(10) REFERENCES vehicule(immat_Vehicule) ON DELETE CASCADE,
-> NCIN_Cli varchar(8) REFERENCES Client(NCIN_Cli) ON DELETE CASCADE,
-> date_loc timestamp default current_timestamp CHECK(date_loc<=now()),
-> duree_loc int(3) check(duree_loc>0),
-> cout_loc decimal(8,3) check(cout_loc>0),
-> CONSTRAINT PK_Location PRIMARY KEY(immat_Vehicule,NCIN_Cli));
Query OK, 0 rows affected (0.11 sec)

mysql> describe location;
```

Field	Type	Null	Key	Default	Extra
immat_Vehicule	varchar(10)	NO	PRI		
NCIN_Cli	varchar(8)	NO	PRI		
date_loc	timestamp	NO		CURRENT_TIMESTAMP	
duree_loc	int(3)	YES		NULL	
cout_loc	decimal(8,3)	YES		NULL	

5 rows in set (0.00 sec)

I.3 Modification de la structure d'une table

❖ Syntaxe de modification de la structure d'une table :

```
ALTER TABLE nom-table  
( ADD [COLUMN] (nom-col type-col [DEFAULT valeur] [contrainte-col])*  
| ADD [CONSTRAINT] (contrainte_table)*  
| MODIFY [COLUMN] nom-col [type-col] [DEFAULT valeur] [contrainte-col]  
| MODIFY [CONSTRAINT] nom_contrainte  
| DROP COLUMN nom-col ,  
| DROP {PRIMARY KEY  
| CONSTRAINT nom_contrainte }  
| RENAME TO <nouveau_nom_table>  
);
```

❖ Syntaxe de suppression d'une table :

```
DROP TABLE nom_table;
```

a) Ajout de colonnes à une table

Ajouter à la table **Client** un champ nommé **Tel_Cli** de type texte et de taille égale à 10.

```
ALTER TABLE Client  
ADD COLUMN Tel_Cli varchar(10);
```

NB
:

l'utilisation de () après le mot clé "COLUMN" est facultatif quand il s'agit de l'ajout d'une seule colonne.

b) Modification d'une colonne de la table

Modifier le type de donnée de la colonne "**Tel_Ci**" de la table "**Client**" en numérique de 8 chiffres.

```
ALTER TABLE Client  
MODIFY COLUMN Tel_Cli INT(8);
```

c) Suppression des colonnes d'une table

Supprimer la colonne "**Adresse_Cli**" dans la table "**Client**".

```
ALTER TABLE Client  
DROP COLUMN Adresse_Cli;
```

d) Ajout d'une contrainte

Vérifier que les valeurs de "**Annee_Acq**" dans la table "**Vehicule**" est entre 1990 et 2008.

```
ALTER TABLE Vehicule  
ADD CONSTRAINT CHECK Annee_Acq BETWEEN 1990 AND 2008;
```

Remarque: Si on veut attribuer la ville "tunis" comme valeur par défaut à tous les clients, il ne s'agit pas d'ajouter une contrainte mais de modifier la colonne "**Ville**"

```
ALTER TABLE Client  
MODIFY COLUMN Ville VARCHAR(20) DEFAULT 'tunis';
```

e) Modification de la clé primaire d'une table

Modifier la clé primaire actuelle de la table Location par la clé primaire composée suivante:

ImmatVehicule, NCIN, Date Loc

```
ALTER TABLE Location  
DROP PRIMARY KEY  
ADD PRIMARY KEY (immat_Vehicule,NCIN_Cli,date_loc);
```

f) Suppression d'une contrainte

Supprimer la clé primaire de la table "Client"

```
ALTER TABLE Client  
DROP PRIMARY KEY;
```

g) Activer/désactiver une contrainte

Désactiver la contrainte de clé primaire de la table "client"

```
ALTER TABLE Client  
DISABLE KEYS;
```

Réactiver cette même contrainte

```
ALTER TABLE Client  
ENABLE KEYS;
```

h) Renommer une table

Renommer la table "Vehicule" en "Voiture"

```
ALTER TABLE Vehicule  
RENAME TO Voiture;
```

i) Suppression d'une table

Supprimer la table "Client"

```
DROP TABLE Client;
```

j) Suppression de la base de données

Supprimer la base de données "Gestion_Location"

```
DROP DATABASE Gestion_Location;
```

I) Introduction

L'opération de mise à jour des tables d'une base de données regroupe :

- la commande d'insertion,
- la commande de modification
- la commande de suppression

II) Mise à jour de données

II.1.1 Insertion de lignes

Syntaxe : Insert Into Nom de table Liste_Nom_Colonne
Values (Liste_Valeur) ;

- Liste_Nom_Colonne : sert à préciser la liste des colonnes dans lesquelles va s'effectuer l'opération d'insertion.

Remarques :

- la liste des colonnes peut être omise dans le cas où l'opération d'insertion concerne toutes les colonnes de la table.

Exemple :

Insert into Clients

Values ('A301','Tounsi','Weal','Tunisienne') ;

Equivalente à

Insert into Clients (COD_CLI, Nom_CLI, Prn_CLI, Nat_CLI)

Values ('A301','Tounsi','Weal','Tunisienne') ;

Equivalente à

Insert into Clients (Nom_CLI , Nat_CLI, COD_CLI, Prn_CLI)

Values ('Tounsi', 'Tunisienne', 'A301','Weal') ;

- Les types de données de la liste_Valeur doivent être compatibles avec ceux de colonnes de la table.
- Unicité de ligne (contrainte de **clé primaire**)
- Valeurs obligatoires associées à une colonne (clause Not Null)
- Contrainte d'intégrité référentielle (cl é étrangère)
- Vérification de la condition de validité **CHECK**)

II.1.2 Suppression de lignes :

Syntaxe : Delete From nom table

Where condition ;

- la clause Where est optionnelle, si elle est absente alors toutes les lignes de la table concernée seront supprimées.

Exemple

Delete from clients  supprime toutes les lignes de la table clients

- La clause **Where** sert à indiquer la condition à vérifier pour les lignes à supprimer.

Exemple

- Si la commande **Delete** ne respecte pas la contrainte d'intégrité référentielle, elle ne peut pas être exécutée.

Exemple1

- Supprimer le fournisseur numéro 707

- Delete from Fournisseur

Where NUm_Frs=707 ;

Le client numéro 707 sera supprimé car il n'existe pas dans la table Articles.

Exemple2

- Supprimer le fournisseur numéro 714
- Delete from Fournisseur
Where Num_Frs = 714 ;

📁 La suppression du fournisseur numéro 714 ne peut pas être réalisée car ce numéro se trouve dans la table Articles (num_frs est une clé étrangère dans la table Articles) et on n'a pas utilisé la clause

« ON DELETE CASCADE » lors de la définition du clé étrangère Num_frs dans la table Articles

create table articles

(num_art integer primary key,

des_art varchar(20) not null,

col_art varchar(20),

pv_art float check(pv_art>0),

pa_art float check(pa_art>0),

num_frs integer references fournisseurs (num_frs));

- ✓ La suppression d'une ligne appartenant à une table donnée peut entraîner la suppression d'autres lignes appartenant à d'autres tables lorsqu'il existe des contraintes d'intégrité référentielle de suppression en cascade (utilisation de la clause **On delete Cascade** dans la définition des clés étrangères).

Exemple

- Supprimer le client ayant le code 'A355'
- Delete from Clients
Where Cod_Cli = 'A355' ;

📁 On remarque que les ventes effectuées par ce client ont été supprimées (toute ligne de la table Ventes ayant Cod_Cli='A355' seront supprimées automatiquement)

II.1.3 Modification de lignes

Syntaxe : Update nom de table

Set Nom_Colonne1= Expression1 [, Nom_Colonne2= Expression2..]

Where condition ;

- La modification peut toucher une ou plusieurs colonnes.
- Si on ne spécifie aucune condition dans la clause **Where**, l'update sera réalisé sur toutes les lignes de la table.
- Les principaux cas pour lesquelles l'ordre **Update** ne peut aboutir sont les mêmes que ceux qui ont été mentionnées pour l'ordre **Insert**.

Exemple :

- * Mettre à jour le prix de vente de l'article numéro 501 de valeur 18.75

Update Articles

Set pv_art = 18.75

Where num_Art = 501 ;

- Mettre à jour l'identité du fournisseur numéro 714 (Nom : « Tounsi » Tel : 98970020)

Update Fournisseurs

Set Nom_Frs = 'Tounsi', tel_frs=989790020

Where num_frs = 714 ;

- Transformer les nationalités des clients en majuscules.

Update clients

Set nat_cli=upper(nat_cli);

- Augmenter le prix de ventes des articles de 18% par apport au prix initial.

Update articles

Set pv_art=(pv_art*118)/100;

III) Recherche de données : requêtes

III.1 Principe

La recherche de données est une opération qui consiste en la recherche de données appartenant à la base.

Une recherche peut s'effectuer en :

- ♦ Une projection sur certaines colonnes d'une table,
- ♦ Une sélection sur certaines lignes d'une table,
- ♦ Une jointure sur deux tables,
- ♦ Toute combinaison de projection, sélection et jointure,

La commande permettant d'effectuer une recherche dans une table de donnée est la commande SELECT.

III.2 La commande **SELECT** :

Syntaxe :

SELECT Nom_champ1, Nom_champ2,...
FROM Nom_Table ;

La consiste à effectuer un sous ensemble de colonnes, d'une table.

- Elle ne concerne qu'une seule table.
- Toutes les lignes relatives aux colonnes feront parti du résultat.

Projection

Syntaxe :

SELECT Nom_champ1, Nom_champ2,...
FROM Nom_Table
WHERE Condition ;

la clause **WHERE** permet d'introduire une condition pour spécifier les lignes à afficher.

- La consiste à afficher un sous ensemble de lignes d'une table.

Sélection

NB : La sélection est souvent combinée avec la projection. Dans ce cas le résultat est un sous ensemble de lignes et de colonnes de cette table.

• Select *

From Nom_Table ;

Cette commande permet d'afficher toutes les colonnes de la table.

Exemple : Afficher la liste des clients

Select * From clients ;

III.2.1) La clause **DISTINCT**:

Rôle: n'afficher qu'un exemplaire de chaque jeu de lignes identiques.

Afficher les nationalités des clients:

Remarque:

- Pour construire une condition, on dispose d plusieurs opérateurs: =, >=, <=, >, <, <> et ! qui est la négation soit !=, !<, !>.
 - Pour combiner plusieurs conditions, utiliser les opérateurs logiques AND, OR et NOT
 - Pour comparer un champ à du texte, on utilisera le caractère « ' » pour encadrer le texte.
- Pour comparer un champ à une valeur numérique, ce caractère ne doit pas être employé.

III.2.2 L'utilisation des attributs Between, In, Like:

Ces attributs permettent de spécifier les conditions de recherche.

- ❖ **Between** : peut être utilisé si une valeur (caractère, date, numérique) se situe entre 2 valeurs.

Exemple : la liste des articles vendus dans la période de 01/01/2007 jusqu'au 8/01/2007.

- ❖ **IN** : vérifie si une valeur de colonne correspond à l'une de celle spécifiée dans la liste.
- ❖ **NOT IN** : vérifie si une valeur de colonne ne correspond pas à l'une de celle spécifiée dans la liste.

Exemple : la liste des magasins de Sfax et gabes.

```
Select * From magasins
Where upper (loc_mag) in ('SFAX','GABES');
Select *
From magasins
Where upper (loc_mag) = 'SFAX' or upper (loc_mag) = 'GABES';
```

Exemple : la liste des magasins autres que de Sfax et gabes.

```
Select *
From magasins
Where upper(loc_mag) not in ('SFAX','GABES');
```

Exemple : la liste des clients tunisiens et algériens

```
Select *
From clients
Where upper (nat_cli) in ('TUNISIEN','ALGERIEN');
```

COD_	NOM_CLI	PRN_CLI	NAT_CLI
A301	NAFTI	SAMIR	TUNISIEN
A360	BEN BRAHIM	HOTHMAN	ALGERIEN
A355	TRABELSI	ABIR	TUNISIEN
A390	LOUKIL	CHOKRI	TUNISIEN
A300	LAJDEL	AYA	TUNISIEN
A377	LAJNEF	KAMEL	TUNISIEN

LIKE : sélectionne les lignes en comparant la valeur d'une colonne de type caractère à une chaîne de caractère.

Remarque : le caractère % représente n'importe quel nombre de caractère.

Le caractère _ représente un seul caractère quelconque.

Exemple : la liste des articles commençant par « Ch »

```
Select *
From articles
Where des_art like 'Ch%';
```

Exemple : la liste des articles commençant par « Ch » et dont le prix de vente est > 70 d

```
Select *
From articles
Where des_art like 'Ch%' and pv_art>70;
```

Exemple : la liste des fournisseurs dont les noms commençant par « Ka » et de taille 6

```
Select *
From fournisseurs
Where nom_frs like 'KA ';
```

III.2.3 L'utilisation d'expressions

Exemple : Calculer la marge bénéficiaire de chaque article (pv_art – pa_art)

```
Select pv_art-pa_art "marge bénéficiaire"  
From articles;
```

Exemple : afficher le prénom et le nom de chaque client séparé par un espace.

```
Select prn_cli||' '||nom_cli "NOM PRtNOM"  
From clients;
```

III.2.4 L'utilisation de fonctions de calcul (fonctions agrégats)

- SQL offre certaines fonctions standard de calcul appelées fonctions agrégats. Ces fonctions ne peuvent être utilisées qu'avec la commande **SELECT** et en dehors de la clause **WHERE**.

- ♦ **COUNT ()** : compte le nombre de ligne
- ♦ **SUM ()** : additionne les valeurs d'une colonne numérique
- ♦ **MIN ()** : trouve la valeur minimale d'une colonne de type caractère ou numérique.
- ♦ **MAX ()** : trouve la valeur maximale d'une colonne de type caractère ou numérique.
- ♦ **AVG ()** : calcule la moyenne des valeurs d'une colonne.

Nombre des fournisseurs

```
Select count (*) "Nombre de Fournisseurs"  
From Fournisseurs;
```

☐ Nombre des clients tunisiens

```
Select count (*) "Nombre des clients Tunisiens"  
From clients  
Where upper (nat_cli) ='TUNISIE';
```

Nombre des clients étrangers :

```
Select count (*) "Nombre des clients étrangers"  
From clients  
Where upper (nat_cli) <>'TUNISIEN';
```

☐ Nombre des articles fournis par le fournisseur numéro 777

```
Select count (*)  
From Articles  
Where num_frs=777;
```

La somme des prix d'achat des articles

```
Select sum(pa_art)  
From articles;
```

☐ La moyenne des prix d'achat des articles

```
Select avg (pa_art)  
From articles
```

```
Select sum(pa_art)/count(*)  
From articles;
```

☐ L'article le prix plus cher (ayant le prix de vente le plus élevé)

Select max (pv_art)

From articles;

☐ La désignation de l'article le prix plus cher (ayant le prix de vente le plus élevé)

Select des_art "Article plus cher"

From articles

Where pv_art=(select max(pv_art)

From articles);

L'article le prix moins cher (ayant le prix de vente le plus bas)

Select min (pv_art) "Prix bas"

From articles;

☐ Le numéro de l'article le moins cher (ayant le prix de vente le plus cher)

Select num_art

From articles

Where pv_art= (select min(pv_art) From articles);

☐ Le numéro de fournisseur qui fourni l'article le plus cher (ayant le prix d'achat le plus cher).

Select num_frs

From articles

Where pa_art= (select max (pa_art) From articles);

☐ Le nombre des clients qui ont acheté des articles dans la période du 01/01/2007 jusqu'à 08/01/2007.

Select count (cod_cli)

From ventes Where dat_vnt between '01/01/2007' and '08/01/2007';

☐ Trouver les désignations des articles dont le prix de vente est supérieur au prix de vente moyen.

Select des_art

From articles Where pv_art > (select avg (pv_art) From articles);

Trouver les informations relatives aux fournisseurs qui n'ont pas fourni des produits (nom et téléphone).

Select nom_frs, tel_frs

From fournisseurs

Where num_frs not in (select num_frs From articles);

☐ Trouver les produits (désignation et couleur) qui ne sont pas encore vendus.

Select des_art, col_art

From articles

Where num_art not in (select num_art From ventes);

III.2.5 Tri des données

La clause **ORDER BY** spécifie la liste des colonnes sur lesquelles l'affichage doit être trié (**ASC** (ascendant) ou **DESC** (descendant)).

• ***C'est le tri par ordre croissant qui est pris par défaut.***

☐ Liste des clients triés par ordre croissant des noms

Select *

From clients

Order by nom_cli;

- ☐ Liste des clients triés par ordre décroissant des prénoms

Select *

From clients

Order by prn_cli desc ;

- ☐ Liste des articles triés par ordre décroissant des prix d'achat et par ordre croissant des prix de vente

Select *

From articles

Order by pa_art desc , pv_art;

III.2.6 Regroupement des enregistrements

a) la clause GROUPBY

Cette clause regroupe les lignes ayant des valeurs identiques dans une ou plusieurs colonnes issues de l'instruction **SELECT**, en une ligne.

Remarque: chaque colonne figurant dans la clause **SELECT** doit figurer dans la clause **GROUP BY**.

- ☐ Nombre des produits fournis par chaque fournisseur

Select num_frs, count (*) "Nombre de produits fournis"

From articles

Group by num_frs;

- ☐ Nombre des produits vendus chaque jour

Select dat_vnt "Date de vente", count (num_art) "Nombre des articles vendus"

From ventes

Group by dat_vnt;

- ☐ Nombre de fois où chaque client a effectué un achat auprès des magasins.

Select Cod_cli , count (*) "Nombre d'achat"

From ventes

Group by Cod_cli;

b) la clause HAVING

Cette clause permet de restreindre la sélection des groupes figurant dans le résultat. Elle précise la condition à laquelle chaque groupe doit répondre.

- ☐ Trouver le nombre des articles achetés par chacun des clients qui effectué deux achats au plus.

Select Cod_cli , count (*) "Nombre d'achat"

From ventes

Group by Cod_cli

Having count (*)>=2;

- ☐ Trouver les articles vendus plus que 2 fois

Select num_art

From ventes

Group by num_art

Having count (*)>2;

III.2.7 Les jointures

- Une jointure réalise une liaison entre deux tables en se basant sur l'égalité entre l'un des attributs de chaque table.
- La condition de jointure doit porter sur les colonnes en commun aux tables utilisées pour l'opération de jointure.

☞ Liste des fournisseurs (nom et numéros de téléphone) de chaque article (num_art).

```
Select num_art,nom_frs, tel_frs  
from fournisseurs,articles  
where fournisseurs.num_frs=articles.num_frs;
```

Remarque : On peut attribuer des préfixes aux noms des tables.

```
Select num_art,nom_frs, tel_frs  
from fournisseurs F,articles A  
where F.num_frs=A.num_frs;
```

☞ Liste des articles vendus (désignation et couleur)

```
Select des_art,col_art  
des_art,col_art from articles A,ventes V  
where A.num_art=V.num_art;
```

☞ Total des ventes pour chaque client.

```
Select cod_cli,sum(qte_vnt*pv_art) "Total des ventes"  
From ventes V,articles A  
Where V.num_art=A.num_art  
Group by cod_cli;
```

☞ Liste des clients ayant un total des ventes supérieur à 1000 D

```
Select cod_cli,sum(qte_vnt*pv_art) "Total des ventes"  
From ventes V,articles A  
Where V.num_art=A.num_art  
Group by cod_cli  
Having sum(qte_vnt*pv_art)>1000;
```


Aide-mémoire

Les formulaires - HTML5

<label> </label>	Etiquetage
--------------------	------------

Number	
Renseigne une valeur numérique	
valeur minimal du compteur	min
valeur maximal du compteur	max
L'avancement du compteur	step
Valeur de départ du compteur	value
Attributs	

Compteur numérique

les données transitent par l'url	get
pour la majorité des cas	post

la redirection du formulaire	
Attribut obligatoire pour une validation en W3C	action
	method

Les Attributs obligatoires

<form method="post" action="traitement.php">	Le code
--	---------

Email : <input type="email" name="email" id="email" required /> <input type="submit" value="ok">	email
Url de votre site : <input type="url" name="url" id="url" />	url
<input type="tel" name="tel" id="tel" />	tel

<input type="number" name="number" id="number" />	
min="5" max="10"	Minimum-max
step="2"	Avancement
range	Curseu

Les input

color	r Saisir une
file	couleur
	Transfert de fichiers
date	Saisir une date
	Mot de passe

Le construire	<form> </form>
Fondamentaux	Regrouper les éléments d'un fomulaire
L'organiser	<fieldset> </fieldset>
	<legend> </legend>

Liste déroulante

select	
selected	sélection par défaut
optgroup	créer des groupes
size	Définit le nombre d'éléments du menu
multiple	choix multiple dans une liste

Zone de texte

<textarea> </textarea>	Zone de texte de plusieurs lignes
	rows
	Nombre de lignes visibles en hauteur de la zone
Attributs	cols
	Nombre de caractères visibles en largeur de la zone

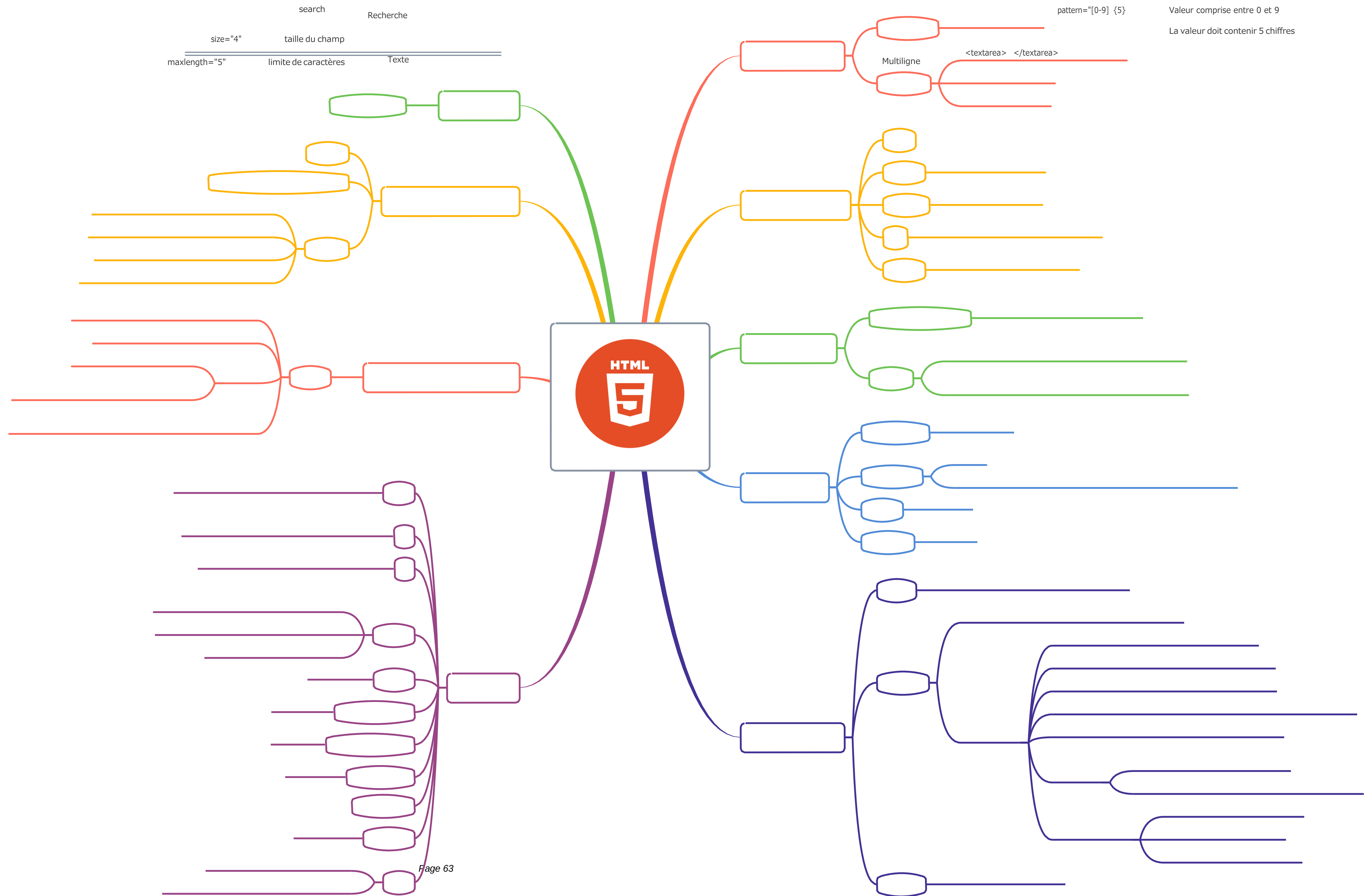
Les boutons

Cases à cocher	cheackbox
	radio
bouton radio	Attributs
	checked
	Présélectionne le bouton radio
Envoyer	submit
Annulation	reset

Libellés	<label for="pseudo"> votre pseudo </label>
	Texte
	<input type="text" name="pseudo">
	size="50"
	nombre de caractères visibles
	maxlength="10"
	caractère maximal autorisé
Monoligne	value="texte"
	Inscrit la valeur dans le champ
	placeholder="votre prénom ici"
	Renseigne sur le contenu à saisir

Zones de saisie

Les attributs	readonly
	Valeur attribuée par default à la ligne
	autofocus
	Donne le focus "la saisie" sur la ligne sélectionnée
	required
	Rend le champ obligatoire à la saisie
	Encodage numérique obligatoire



Les couleurs et les arrière-plans

Les Bordures

La propriété CSS border

Tous les éléments HTML acceptent une bordure Css, que cela soit des images, des div, des span ou des éléments de formulaires

Les bordures Css se situent entre les marges intérieures appelées padding et les marges extérieures appelées margin



On peut définir globalement les quatres bordures Css avec la propriété Css border si les bordures Css sont identiques.

Exemple
h1{ border : #FF00FF solid 2px ;
/* Définir toutes les bordures ensemble */}

Les différentes bordures d'un élément

1 border-top :la bordure Css haute

border-top : #FF00FF solid 2px;
/* Définition de la bordure Css haute */

2 border-bottom :la bordure Css basse

border-bottom : #FF00FF solid 2px;
/* Définition de la bordure Css basse */

3 border-left :la bordure Css gauche

border-left : #FF00FF solid 2px;
/* Définition de la bordure Css gauche */

4 border-right :la bordure Css droite

border-right : #FF00FF solid 2px;
/* Définition de la bordure Css droite */

la couleur de la bordure

La propriété Css border-color permet de modifier la/les couleurs sur l'ensemble des bordures Css

Exemple
.nav{ border-color : crimson rgb(255,0,0) hsl(16,100%,50%) #FF00FF ; }

Les propriétés Css border-[left|right|top|bottom]-color permettent de modifier la couleur d'une bordure Css précise

Exemple
.nav { border-top-color : crimson;
/* Couleur de la bordure Css haute */}

l'épaisseur de la bordure

La propriété Css border-width permet de modifier la/les tailles sur l'ensemble des bordures Css

Exemple
.nav { border-width : 1px 2px 3px 4px; }

Les propriétés Css border-[left|right|top|bottom]-width permettent de modifier la taille d'une bordure Css précise

Exemple
.nav { border-left-width : 4px;
/* Taille de la bordure Css gauche */}

le style de la bordure

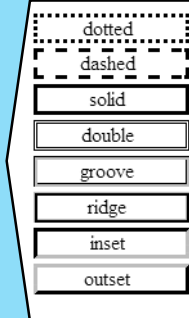
La propriété Css border-style permet de modifier le/les styles sur l'ensemble des bordures Css.

Exemple
.nav{border-style : dotted ; }

Les propriétés Css border-[left|right|top|bottom]-style permettent de modifier le style d'une bordure Css précise.

Exemple
.nav{ border-top-style : dotted;
/* Style de la bordure Css haute */}

Formes des styles



La Couleur d'avant-plan

La propriété CSS color permet de modifier la couleur d'avant-plan .

On peut définir les couleurs avec des valeurs hexadécimales comme (#ff0000), ou avec les noms des couleurs ("red"), ou avec des valeurs RGB (rgb(255,0,0)).

Exemple
h1 {
color: #ff0000;
}

La Couleur d'arrière-plan

La propriété background-color décrit la couleur d'arrière-plan des éléments.

Pour changer la couleur d'arrière-plan d'une page entière, il faudrait donc appliquer la propriété 'background-color' à l'élément <body>.On peut aussi appliquer une couleur d'arrière-plan à d'autres éléments y compris les titres et le texte.

Exemple
body {
background-color: #FFCC66;
}
h1 {
color: #990000;
background-color: #FC9804; }

L'image d'arrière-plan

La propriété CSS background-image sert à insérer une image d'arrière-plan.

Exemple
body {
background-image: url("image.gif");
}

background-repeat

Background-repeat: repeat-x;
L'image se répète horizontalement

background-repeat: repeat-y;
L'image se répète verticalement

background-repeat: repeat;
L'image se répète horizontalement et verticalement

background-repeat: no-repeat;
L'image ne se répète pas

background-attachment

La propriété background-attachment définit si l'image d'arrière-plan est fixe ou bien défile avec l'élément conteneur.

Background-attachment: scroll;
L'image défile avec la page (non bloquée)

Background-attachment: fixed;
L'image est bloquée

background-position

Par défaut, l'image d'arrière-plan se positionnera dans le coin supérieur gauche de l'écran. La propriété background-position permet de changer cette position.

Les coordonnées peuvent s'exprimer en pourcentages , ou en unités fixes (pixels, centimètres,etc.), ou on peut utiliser les mots-clés "top", "bottom", "center", "left" ou "right"

Exemple
body {
background-color:#FFCC66;
background-image: url("image.gif");
background-repeat: no-repeat;
background-attachment: fixed;
background-position: right bottom;
}

La transparence

La propriété CSS opacity rend transparent les éléments d'une même zone

Exemple
h1{
color: white;
opacity: 0.3;
}

Le fond dégradé

Linéaire
background: linear-gradient

Circulaire
background: radial-gradient

Exemple
body {
background: radial-gradient(red, blue);}

Combiner des valeurs

On peut obtenir le même résultat avec background en une seule ligne de code

[background-color] | [background-image] | [background-repeat] | [backgroundattachment] | [background-position]

background: #FFCC66 url("butterfly.gif") no-repeat fixed right bottom;